MAC-Gyver: Open, Programmable, Scheduling for AI-RAN 6G Systems

Maxime Elkael, Reshma Prasad, Tamerlan Aghayev, Salvatore D'Oro, Michele Polese, Tommaso Melodia

arXiv:2607.26012v3 [cs.NI] 8 Aug 2026

***Abstract*—Cellular networks are integrating Artificial Intelligence (AI) into radio access network control. The MAC scheduler is a promising target because it allocates a limited resource, spectrum, at every slot, under competing latency, throughput, and reliability requirements. However, most learning-based schedulers are evaluated only in simulation. Production schedulers are difficult to modify, and realistic stress tests require more radio hardware than most laboratories can provide. We present MAC-Gyver, an open-source framework for developing and evaluating scheduling applications that execute directly inside the OpenAirInterface scheduler. It exposes scheduler observations and controls through typed interfaces while preserving the underlying protocol and real-time execution paths. The same applications run over the air and in `mac-emu`, a PHY-less emulator that executes the production OpenAirInterface Layer 2 stack for up to 90 users on one host at real-time slot pace, with a 3GPP-compliant channel model. To showcase the flexibility of MAC-Gyver, we evaluate two use cases. A proactive Uplink (UL) scheduler predicts packet arrivals and roughly halves median round-trip latency. A frequency-selective UL scheduler selects contiguous sub-bands from per-PRB sounding observations and is evaluated across mobility and power-limited operating points against an offline scheduling ceiling. Together, they show how the same production stack can be an AI playground that supports implementation, controlled evaluation, and Over-the-Air (OTA) validation through complementary scheduling use cases.**

***Index Terms*—5G, 6G, AI-RAN, MAC Scheduler, OpenAirInterface, dApp, Network Emulation.**

I. Introduction

Research on 6th generation (6G) cellular systems increasingly considers Artificial Intelligence (AI) as part of the Radio Access Network (RAN) design. This is a key principle within the AI-RAN Alliance, which distinguishes three forms of integration: (i) AI-and-RAN, where AI and RAN workloads share infrastructure; (ii) AI-on-RAN, where the RAN serves edge AI workloads; and (iii) AI-for-RAN, where AI improves RAN operation and its Key Performance Indicators (KPIs) [1].

We study AI-for-RAN at the Medium Access Control (MAC) layer. At every slot, the scheduler selects the users to serve, assigns spectrum, and enforces the Quality of Service (QoS) requirements of traffic ranging from high-throughput video to Ultra Reliable and Low Latency Communications (URLLC) [2, 3]. These decisions trade throughput and fairness against latency and reliability on a sub-millisecond timescale, while considering scarce resources, i.e., bandwidth, and managing user mobility, Multiple Input, Multiple Output (MIMO), and other requirements [3–5]. Consequently, many studies apply AI and Machine Learning (ML) to MAC scheduling, but most evaluate their algorithms through analysis or simulation [6]. Only a small subset reaches a production stack [7–9].

This gap is primarily an engineering problem. In open platforms such as OpenAirInterface (OAI) and OCUDU, the scheduler is intertwined with the protocol stack. Implementing a new policy requires understanding thousands of lines of C/C++ together with their timing and thread interactions. Observability is also coarse: signals such as per-Physical Resource Block (PRB) Signal-to-Noise-Ratio (SNR) require further modifications to the stack. These costs keep most learning-based schedulers in simulation, where protocol overhead, timing constraints, and radio impairments are abstracted and necessarily approximated.

To address these limitations, we introduce MAC-Gyver, an open and programmable MAC-layer framework. First, MAC-Gyver decomposes the scheduler into a fixed pipeline of typed policy stages. A researcher implements a scheduling policy as a Layer 2 Distributed Application (dApp) [8, 10] and selects it from the Next Generation Node Base (gNB) configuration at boot. The dApp executes synchronously in the scheduler and reads the same state as the default policy. For policies that operate across scheduling opportunities, MAC-Gyver additionally lets dApps observe transmission outcomes, and retain persistent context. Second, `mac-emu` replaces the radio with a 3GPP channel model while preserving the protocol stack through Layer 2 (L2). We implement MAC-Gyver in OAI, and show how it runs one gNB and up to 90 concurrent User Equipments (UEs) on a single host in real-time. Thanks to this design choice, an Over-the-Air (OTA) testbed with real radios and `mac-emu` use the same scheduling interfaces, policies, and event logs. Thus, any policy developed in MAC-Gyver can be ported to the OTA network without re-implementation. The testbed provides real channels and commercial devices; `mac-emu` provides controlled channels, repeatable experiments, and cell-scale contention. The connection across emulation and real-world deployment is an advantage compared to traditional discrete-event simulators (e.g., ns-3 [11]), as they only evaluate schedulers in simulated time. In this way, MAC-Gyver provides a programmable framework for L2 prototyping and bridges the gap between research and real-world demonstrations.

The contributions of this paper are as follows:

- We decompose the per-slot Uplink (UL) and Downlink (DL) scheduler into typed policy stages that share one candidate

The authors are with the Institute for Intelligent Networked Systems, Northeastern University, Boston, MA, USA. E-mail: {m.elkael, r.prasad, aghayev.t, s.doro, m.polese, melodia}@northeastern.edu.

This work was supported by the U.S. NSF under award TI-2449452 and by OUSW(R&E) through Army Research Laboratory Cooperative Agreement Number W911NF-24-2-0065. The views and conclusions contained in this document are those of the authors and should not be interpreted as representing the official policies, either expressed or implied, of the Army Research Laboratory or the U.S. Government. The U.S. Government is authorized to reproduce and distribute reprints for Government purposes notwithstanding any copyright notation herein.

structure. The stages expose channel observations, buffer state, Hybrid Automatic Repeat reQuest (HARQ) feedback, and the decisions needed for online control and offline analysis. MAC-Gyver also lets dApps observe transmission outcomes, and retain persistent context across scheduling opportunities. We have implemented this design on the OAI scheduler, and it has already been contributed upstream [12].
- We develop `mac-emu`, a Physical (PHY)-less emulator that runs one OAI gNB and up to 90 UEs on one host in real-time. It uses the same policies and logs as the OTA deployment.
- Thanks to MAC-Gyver's dApp-based architecture, we demonstrate stateful, closed-loop scheduling with a proactive UL scheduler that learns packet arrivals online and schedules grants before the UE requests them. This application was presented in our AUGUSTE conference article [13], of which this paper is an extension. In OTA experiments with commercial UEs, it reduces median Round Trip Time (RTT) by about 50% (20 ms to 10 ms) and one-way UL latency from 20 ms to 7 ms, at 7% overhead.
- We use MAC-Gyver and `mac-emu` to design and evaluate a frequency-selective UL scheduler from per-PRB sounding observations that improves aggregate throughput by up to 7%, with gains determined by channel freshness and the users' power-limited operating point.

The remainder of this paper is organized as follows. Section II presents the design tradeoffs and the scheduler design, while Sec. III presents `mac-emu`. Sections IV and V evaluate the two applications. Section VI reviews related work, and Sec. VII discusses the lessons learned and concludes.

II. MAC-Gyver Scheduler Architecture

In this section, we first discuss the constraints and tradeoffs involved in scheduler design. We then use these to motivate MAC-Gyver's scheduler architecture, designed to make scheduling policies easy to modify and extend.

A. Scheduler Design Constraints

At each scheduling opportunity (i.e., slot), the scheduler decides which UEs to serve and selects their transmission parameters such as the Modulation and Coding Scheme (MCS), the allocated PRBs, the Time-Domain Allocation (TDA) (i.e., which Orthogonal Frequency Division Multiplexing (OFDM) symbols), the beamforming configuration, and the transmit power. Scheduling is therefore a real-time problem, solved every 500 μs at 30 kHz Subcarrier Spacing (SCS), over an action space with several decision variables for each UE.

These decision variables are coupled, both with each other and across slots. For example, the UE's power budget ties the number of allocated PRBs to the achievable MCS within a slot, while across slots an UL grant must be announced on the Physical Downlink Control Channel (PDCCH) during an earlier DL slot, so the configured K_2 values, the Time Division Duplexing (TDD) pattern, and the available control-channel resources determine which future UL slots are reachable. Further constraints, e.g., contiguous UL PRB allocation and the timing of HARQ retransmissions and their Physical Uplink Control Channel (PUCCH) feedback, restrict the feasible schedules.

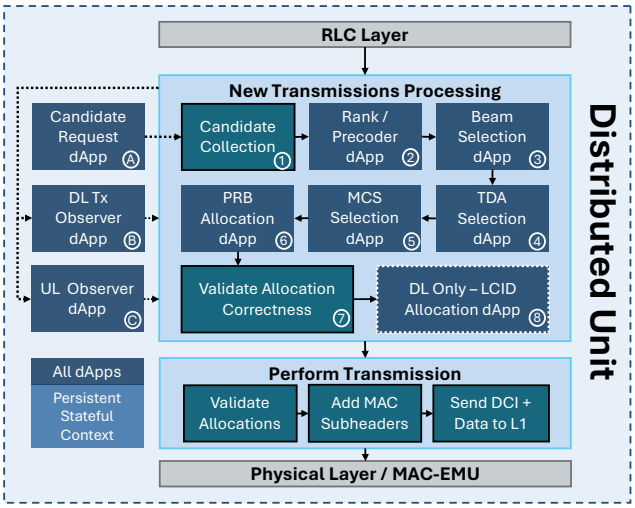


Fig. 1: MAC-Gyver dApp architecture. Numbered decision dApps form the scheduling pipeline within one opportunity; lettered dApps observe transmission events or request candidates across opportunities. All dApps may access a persistent stateful context.

Consequently, formulations that optimize only data-channel PRBs can produce allocations that the cellular stack cannot use when considering the constraint of the 3GPP NR protocols.

These characteristics make schedulers hard to design while maintaining flexibility and integrating AI/ML routines. Production implementations, such as the existing OAI scheduler, entangle every scheduling decision with the management of the aforementioned constraints and of the real-time path. Therefore, changing a single decision, e.g., how the MCS is selected, requires understanding and modifying logic across the whole scheduler. At the other extreme, rewriting a scheduler from scratch gives full flexibility, but it means re-implementing the protocol handling, with the risk of re-introducing edge cases and bugs that production code has long since fixed. We therefore propose a modular architecture, which isolates each decision behind a replaceable interface while sharing the feasibility checks and the real-time execution path. We describe it in the next subsection.

B. Architecture

MAC-Gyver designs and implements the MAC scheduler as a staged pipeline (Fig. 1), where each stage makes one scheduling decision, e.g., MCS selection or PRB allocation. Stages exchange their inputs and decisions through a shared per-UE candidate structure. Each stage is a replaceable policy. At gNB startup, the configuration selects one implementation per stage and composes a complete scheduler from individual modules, which can be abstracted as dApps. Beyond these stages, MAC-Gyver lets dApps observe transmission outcomes and request additional UL candidates. All dApps may access a persistent stateful context to carry application-defined state across scheduling opportunities. The pipeline order, the candidate structure, and the protocol feasibility checks remain fixed, so dApps change selected scheduling decisions without reimplementing the surrounding protocol handling.

Figure 1 shows the pipeline that MAC-Gyver executes at each scheduling opportunity both in DL and UL. Candidate

collection (1) populates one candidate structure per schedulable UE (Listing 1), i.e., a UE with a pending HARQ retransmission or with new data to send and a free HARQ process. The dApps then decide the MIMO rank and which precoder to use (2), the serving beam (3), the TDA [1](4), the MCS (5) and the PRB allocation (6). Retransmission candidates traverse the same dApps, but their decisions must preserve the original transport-block size, e.g., a TDA, MCS or PRB allocation change is accepted only if it reproduces the same block size. In deployments without beamforming, the beam dApp returns a single default beam and the pipeline operates unchanged. PRB allocation may also lower the selected MCS to satisfy the UE power budget. The validation step (7) then checks each grant against the protocol constraints before dispatch. Finally, the Logical Channel Identifier (LCID) dApp (8) divides the transport-block bytes among logical channels for the UE on the DL only. This multiplexing is a sender-side decision, as the UE MAC performs the same task on the UL path. The dApps receive their input data per UE from the candidate data structure of Listing 1 (`nr_ul_candidate_t` is the UL analog). They also output their controls in that same structure. Candidate collection fills the metrics in that structure, and decision fields are written by the dApps, which ensures every dApp observes the same state, enriched by the previous dApps' decisions.

Beyond the numbered stages, Fig. 1 shows three dApp interfaces for policies that operate outside of the mandatory scheduling chain. The candidate-request dApp (A) runs during UE candidate collection and may request that a UE enter the pipeline before its buffer state indicates pending data (this enables e.g., the proactive scheduler of section IV). The DL transmission observer (B) is invoked after a scheduled transport block has been constructed, while the UL observer (C) receives the outcome of each UL grant. These two observers enable MAC-Gyver to expose data such as the amount of padding bytes in a received UL Packet Data Unit (PDU) or the time of arrival of DL PDUs. Unlike the numbered stages, these observers do not modify the completed transmission. Instead, their primary role is to expose extra data to enable more intelligent decisions. Finally, note that all dApps have access to a stateful context, which is (i) shareable across the different dApps and (ii) persisted across slots. This enables dApp designers to expose context in new ways and to make dApps stateful if needed (for example, combined with dApp (B), it enables tracking the rate of arrival of DL PDUs).

In its OAI implementation, MAC-Gyver passes each candidate scheduling structure through OAI's existing grant-validation path. This path checks, for example, that the selected PRBs are available, that the PDCCH has sufficient capacity for the grant, and, on the DL, that a PUCCH resource is available for HARQ feedback. When a check fails, the pipeline rejects the grant and reports the cause through the OAI logs, helping the researcher identify and correct infeasible behavior in the corresponding dApp. Only validated grants are committed and sent to Layer 1 through the Functional Application Platform Interface (FAPI) interface.

[1]in 5G, the TDA indicates to the UE which OFDM symbols it is allowed to use in the associated grant

```c
typedef struct nr_dl_candidate {
  /* Identity and scheduling state */
  uint16_t rnti;
  bool     is_retx;         // HARQ retx pending
  uint32_t pending_bytes;   // RLC buffer occupancy
  float    avg_throughput; // EWMA goodput
  float    bler;            // current BLER estimate
  int      current_mcs, max_mcs;
  uint64_t fiveQI;          // QoS class
  nssai_t  nssai;           // slice ID
  bool     skipped, scheduled; // stage outcomes
  /* Channel observations (read-only) */
  uint16_t cqi;             // wideband CQI
  uint8_t  csi_ri;          // reported rank
  const int16_t *beam_rsrp;     // per-SSB L1-RSRP
  const float *srs_rb_energy;  // per-RB channel
                               // energy from SRS
  /* Decisions, written by the named dApp */
  NR_sched_pdsch_t sched_pdsch; // MCS, PRBs,
                                // layers, TDA
  int alloc_beam_idx;       // selected beam
  int alloc_cce_index;      // PDCCH position
} nr_dl_candidate_t;
```

Listing 1: DL candidate struct.

MAC-Gyver's scheduler backbone has been upstreamed to OAI [12], and the complete MAC-Gyver and `mac-emu` source code is available as open source [2]. The same interfaces also support the agentic scheduler control demonstrated in AgentRAN, which builds on MAC-Gyver's lightweight design and scheduler-state exposure [14].

The two use cases exercise complementary parts of this architecture. Section IV combines the candidate-request dApp (A), DL transmission observer (B), and UL observer (C) with persistent context to act across scheduling opportunities. Section V uses the rank and precoder produced by dApp (2) to implement frequency-selective PRB allocation in dApp (6); it also replaces MCS selection (5) independently in the oracle experiment. Before presenting these use cases, the next section introduces `mac-emu`, which preserves these interfaces while providing controlled execution of the complete scheduler.

III. MAC-EMU: Real-Time L2 Emulation at Scale

Next, we present `mac-emu`, MAC-Gyver's software-emulation component whose role is to execute the same OAI scheduler and registered dApps under controlled channel conditions and at cell scale, without requiring radio hardware. `mac-emu` preserves the protocol stack through L2 and follows the real-time slot clock, so policies exercise the same OTA scheduling state, protocol constraints, and execution path.

A. Motivation

Exposing the scheduler through MAC-Gyver addresses the difficulty of implementation, but evaluation still requires representative channel behavior, cell-scale contention, and repeatable operating conditions. While OTA and hardware-based testbeds provide high-fidelity evaluation, they lack the scale of real deployments, i.e., tens to a hundred RRC-active UEs [15]. Simulators such as ns-3 [16] provide scale by executing a

[2]https://github.com/wineslab/MAC-Gyver

reimplemented Distributed Unit (DU) in simulated time and cannot directly host arbitrary applications and traffic. Neither option alone provides controlled, real-time, cell-scale execution of the exact policy deployed in the DU.

In the current research landscape, scaled real-time experiments happen on hardware based testbeds (e.g., X5G [17], Colosseum [18], OAIC [19]). This gives a great degree of realism, but it comes at the cost of complexity in the infrastructure management and experiment execution. Indeed, managing multiple physical nodes (either for RAN or UEs) requires a high level of automation, multiple servers, and software that is distributed across the network. This complexity, combined with use cases that inherently push the boundaries of previous experiments, forces researchers to often redesign their own harness and scripts for new experiments, which requires them to manage edge cases and carefully handle all the potential states of the experiments. All this complexity compounds due to the mix of software, hardware and networking elements involved, and with the fact that lab deployments usually are temporary, making reproducibility challenging. For these reasons, software-based experimentation should complement hardware-in-the-loop validation. Software platforms enable rapid, reproducible, and scalable exploration, while hardware remains important for validating selected policies under real radio impairments. Executing the same protocol stack and policy in both environments, as enabled by MAC-Gyver, makes this transition meaningful by avoiding differences caused by reimplementation and subtle differences between simulators and real cellular stacks.

One such tool is OAI's RF Simulator (RFsim). It runs the real OAI stack fully in software, supports application traffic, and can be packaged as a reproducible single-host experiment (via, e.g., Docker Compose). It retains the complete PHY and exchanges In-Phase/Quadrature (IQ) samples between the gNB and software UEs via TCP. This makes RFsim a good tool for protocol-level evaluation. However, retaining waveform processing also preserves its computational cost, and the use of TCP makes it non real-time.

Prior work has characterized these limitations. Wei *et al.* show that PHY processing dominates the computational cost of an OAI gNB, with the DU accounting for most of the cost in a disaggregated deployment [20]. Rouili *et al.* report that RFsim consumes more CPU than their SDR-based deployment and attribute this overhead to PHY processing and the exchange of IQ samples [21]. More recently, Mamaghani *et al.* benchmark an RFsim-based system with up to five UEs on an AMD Ryzen 9 7900X host with 32 GB of memory [22]. After extending RFsim with multi-tap channel convolution, they report a 90th-percentile slot-computation time of 8 ms for five UEs; beyond four UEs, round-trip latency frequently reaches hundreds of milliseconds and connections may fail. These results show that retaining waveform-level processing becomes a bottleneck well before the cell-scale experiments targeted in this work.

`mac-emu` therefore removes waveform-level PHY processing while retaining the protocol layers and scheduling path that determine dApp behavior. This boundary provides the scale of a software abstraction without replacing the DU scheduler under evaluation. The remainder of this section describes the resulting architecture, channel model, and validation.

B. Architecture

`mac-emu` runs the production gNB and UE stacks unmodified down to their FAPI boundary, and replaces everything below it (i.e., the PHY). The runtime replaces the PHY by synthesizing the Layer 1 observations and transmission outcomes presented to the scheduler, abstracting the underlying PHY implementation, such as waveform generation and Radio Frequency (RF) transmission. As shown in Fig. 2, a thin adapter bypasses the PHY on each side and exchanges the FAPI messages with the `mac-emu` runtime over UDP, while Radio Resource Control (RRC), Packet Data Convergence Protocol (PDCP), Radio Link Control (RLC), and MAC run the deployed OAI code. Because MAC-Gyver and its dApps execute above this boundary, the same dApp interfaces and policy implementations run in `mac-emu` and over the air; only the source of Layer 1 observations and transmission outcomes changes.

Inside the runtime, a software slot clock paces the system and fires once per slot (500 μs at the 30 kHz SCS used in this work) and distributes slot ticks to the gNB and every UE. At each tick, DL Downlink Control Informations (DCIs) and data flow from the gNB adapter to the UEs, while UL data and HARQ feedback flow back. Both directions traverse the channel model: for every scheduled transport block, it decides the ACK/NACK outcome that enters the feedback stream, and it synthesizes the Layer 1 measurements that a real PHY would report to the scheduler (purple arrow in Fig. 2). Section III-C details this model.

Real-time execution rests on three mechanisms. First, the slot clock runs on a dedicated thread with real-time priority, pinned to an isolated core, and paces itself on absolute deadlines. It does so by sleeping until shortly before the next slot boundary, and by busy-spinning for the remainder, avoiding wake-up jitter. Second, the channel model executes asynchronously, on lower-priority worker threads that cannot preempt the thread dedicated to the slot clock. This means that the critical path stays free for message dispatch and scheduling. Third, each UE runs as a separate OAI softUE process whose Layer 1 is bypassed by the UE adapter, so a single host executes the gNB and tens of UEs concurrently. Section III-D quantifies the resulting deadline behavior at up to 90 UEs. We observe no computational bottleneck at this count; rather, 90 is the maximum supported by OAI in our 40 MHz configuration because of its current PUCCH-resource allocation.

C. Channel Model

`mac-emu` abstracts waveform processing but must reproduce its consequences on the scheduler. For each scheduled transport block, its configurable channel model determines the ACK/NACK outcome returned through the protocol stack, and synthesizes the Layer 1 measurements exposed to the scheduler and its dApps. The model is defined per UE in a YAML file that specifies the path-loss model, shadowing, small-scale fading, antenna gains, and mobility. We implement free-space, Urban Micro-cell (UMi), and Urban Macro-cell (UMa) path loss as well as TDL and CDL fading from the 3GPP channel model

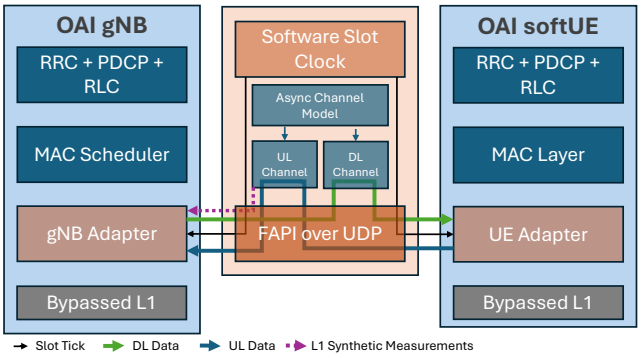


Fig. 2: `mac-emu` architecture. Unmodified OAI L2 stacks run to their FAPI boundary; adapters and the `mac-emu` runtime bypass the PHY. The software slot clock paces the gNB and the UE processes; the purple arrow carries the Layer 1 measurements synthesized by the channel model.

technical report 38.901 [23]. Mobility combines a Doppler frequency with stateful patterns, e.g., a UE that alternates between moving and standing still. As the system is focused on TDD, we assume that DL and UL channels are reciprocal.

The ACK/NACK decision proceeds in three steps. First, a link budget and the fading realization give per-subcarrier SNRs,

$$\gamma_k(t) = \bar{\gamma}\,|H_k(t)|^2, \tag{1}$$

where $\bar{\gamma}$ is the average SNR from transmit power, antenna gains, path loss, shadowing, and thermal noise, and $H_k(t)$ is the channel frequency response at subcarrier k. Second, Exponential Effective SINR Mapping (EESM) compresses the allocated subcarriers into one effective SNR [24],

$$\gamma_{\text{eff}} = -\beta_m \ln\Big(\tfrac{1}{N}\sum_{k=1}^{N} e^{-\gamma_k/\beta_m}\Big), \tag{2}$$

where N is the number of allocated subcarriers and β_m is the calibration factor of MCS m from [24]. HARQ retransmissions add their effective SNRs in the linear domain (Chase combining). Third, a three-parameter Richards curve maps the combined SNR to an error probability,

$$\text{BLER}_{m,w}(\gamma) = \big(1 + e^{a(\gamma-b)}\big)^{-c}, \tag{3}$$

where (a, b, c) control the steepness, position, and asymmetry of the waterfall for MCS m and number w of allocated PRBs; the outcome is one Bernoulli draw per transport block.

Parameters (a, b, c) are fitted offline for each link direction (DL or UL), MCS, and number of PRBs allocated to the transmission. We do this using link-level simulations of the OAI UE and gNB receivers. OAI exposes each receive chain in isolation as a link-level simulator, `nr_dlsim` for the UE's Physical Downlink Shared Channel (PDSCH) receiver and `nr_ulsim` for the gNB's Physical Uplink Shared Channel (PUSCH) receiver: each run encodes transport blocks, applies Additive White Gaussian Noise (AWGN), executes the full demodulation and Low-Density Parity-Check (LDPC) decoding chain, and reports the block error rate. We sweep SNR for the AWGN channel and fit Eq. (3) to the resulting waterfall. Each possible number of PRBs requires its own curve because it sets the transport-block size and therefore the LDPC code-block segmentation, which changes both the position and the steepness of the waterfall. We run the parameter sweep offline for a given carrier configuration, producing one curve per direction, MCS, and number of allocated PRBs. For the 40 MHz carrier used in this work (106 PRBs and 29 MCS indices per direction), it yields 6,148 curves.

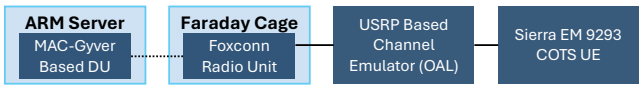


Fig. 3: OAL hardware reference setup.

By construction, these curves reproduce the LDPC receivers exactly. A real link, however, deviates from them due to radio-frequency impairments, such as transmitter error-vector magnitude and phase noise, as well as implementation differences between commercial receivers and OAI (e.g., quantization in the ASIC of a UE).

We use the hardware reference setup in Fig. 3 to calibrate and validate `mac-emu`. An OAI gNB drives a Foxconn RPQN Radio Unit (RU), which connects through the OpenAirLink (OAL) programmable channel emulator [25] to a Sierra Wireless EM9293 Commercial Off-the-Shelf (COTS) UE. OAL applies controlled attenuation and 3GPP fading profiles to the bidirectional radio link. Measurements from this setup parameterize hardware-specific effects in the channel model, while Sec. III-D uses the same physical chain for closed-loop validation. Note that at the time of writing, our OAL setup only supports a 100 MHz (273-PRB) cell configuration, while `mac-emu` experiments are at 40 MHz (106-PRB) throughout this work. We consider 106 PRBs allocations throughout this section, which equalizes transport-block sizes.

We take long fixed-MCS measurements at SNRs well above each waterfall, with HARQ retransmissions disabled so that every logged block is an initial transmission. Where the fitted curves predict a vanishing Block Error Rate (BLER), the hardware instead settles on an error floor. Figure 4 tests whether this floor depends on either MCS or the number of allocated PRBs. Panel (a) varies the MCS over indices 0–14 while capping each allocation at 106 PRBs. After retaining the stationary half of each run, the resulting 9.6 million blocks yield an average error floor of 1.21×10^{-5} with no systematic dependence on MCS. We restrict this experiment to the contiguous range 0–14 because, at higher indices, the setup did not consistently provide stationary measurements sufficiently above the decoding waterfall. Panel (b) fixes MCS 8 and varies the allocation from 20 to 106 PRBs. Across 7.2 million blocks, the average is 1.16×10^{-5}, with no dependence on the number of allocated PRBs. We therefore set $\varphi = 1.2 \times 10^{-5}$ and extend the ACK/NACK decision as

$$P(\text{NACK}) = \varphi + (1-\varphi)\,\text{BLER}_{m,w}\big(\gamma_{\text{eff}}\big), \tag{4}$$

Beyond this data-plane focused model, we also model the effect of channel impairments on the control channel that carries the DL HARQ feedback. Our experiments reveal that, on real hardware, the gNB occasionally fails to detect feedback that the UE did transmit, which the gNB conservatively treats as a NACK, scheduling a retransmission. We measure this effect over three 3GPP TDL fading profiles of increasing delay spread

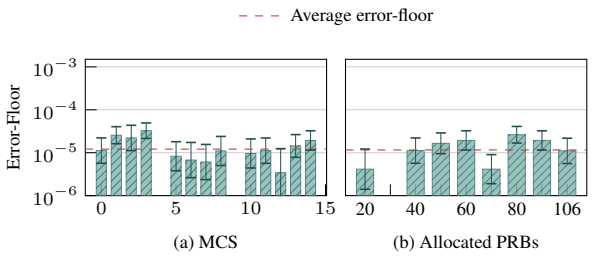


Fig. 4: High-SNR initial-transmission error floor across (a) MCS at 106 PRBs and (b) the number of allocated PRBs at MCS 8. Bars aggregate the retained stationary measurements; error bars show 95% Wilson intervals, and dashed lines show the average error floor in each panel.

(TDL-A30, TDL-B100, and TDL-C300, i.e., 30 to 300 ns of delay spread [23]), reproduced by the channel emulator: the rate of missed detection is strongly ordered by the delay spread, at 0.06%, 0.77%, and 5.5% of feedback occasions, respectively.

The delay-spread dependence follows from the encoding of the HARQ feedback, which goes through PUCCH format 0. PUCCH format 0 encodes ACK/NACK as cyclic shifts of a base sequence. The detector therefore discriminates hypotheses in the delay domain and tolerates a bounded delay error around the expected arrival time. For feedback occasion i, `mac-emu` computes the instantaneous power-weighted delay centroid

$$\bar{\tau}_i = \frac{\sum_{\ell=1}^{L} |h_\ell(t_i)|^2 \tau_\ell}{\sum_{\ell=1}^{L} |h_\ell(t_i)|^2}, \tag{5}$$

where L is the number of channel taps, $h_\ell(t_i)$ is the complex coefficient of tap ℓ at time t_i, and τ_ℓ is its delay.

We use a receiver-level abstraction of the UL timing-tracking loop. The gNB estimates each UE's UL arrival time and sends timing-advance commands, which the UE applies to subsequent transmissions. Since the loop retains state, a residual timing error remains at the gNB's PUCCH detector and evolves across feedback occasions. `mac-emu` represents this correlated error with the Gauss–Markov state n_i. The state and the resulting Discontinuous Transmission (DTX) decision are

$$n_i = \rho_i n_{i-1} + \sqrt{1-\rho_i^2}\,\epsilon_i, \qquad \rho_i = e^{-\Delta t_i / t_{\mathrm{corr}}}, \tag{6}$$

$$d_i = \mathbf{1}\{\bar{\tau}_i + \sigma n_i > W\}, \tag{7}$$

where n_i is the unit-variance Gauss–Markov timing-error state, $\epsilon_i \sim \mathcal{N}(0,1)$ is a new Gaussian random sample drawn at feedback occasion i, independently of the previous samples, Δt_i is the time since the previous feedback occasion, and t_{corr} is the tracking-loop correlation time. The parameter σ scales the timing error, W is the detector window, and $d_i = 1$ indicates a DTX. The stationary distribution of n_i is independent of t_{corr}. Consequently, W and σ determine the DTX rate, while t_{corr} determines how events cluster in time.

We calibrate the model against the DTX statistics measured on the OAL-based testbed: W and σ by maximum likelihood on the per-channel DTX rates, and t_{corr} on the burstiness of the resulting retransmissions. We obtain $W = 475$ ns, $\sigma = 140$ ns, and $t_{\mathrm{corr}} = 100$ ms, which yields DTX rates of $0.07\%/0.28\%/5.4\%$ on TDL-A30/-B100/-C300, against the $0.06\%/0.77\%/5.5\%$ measured on the hardware testbed.

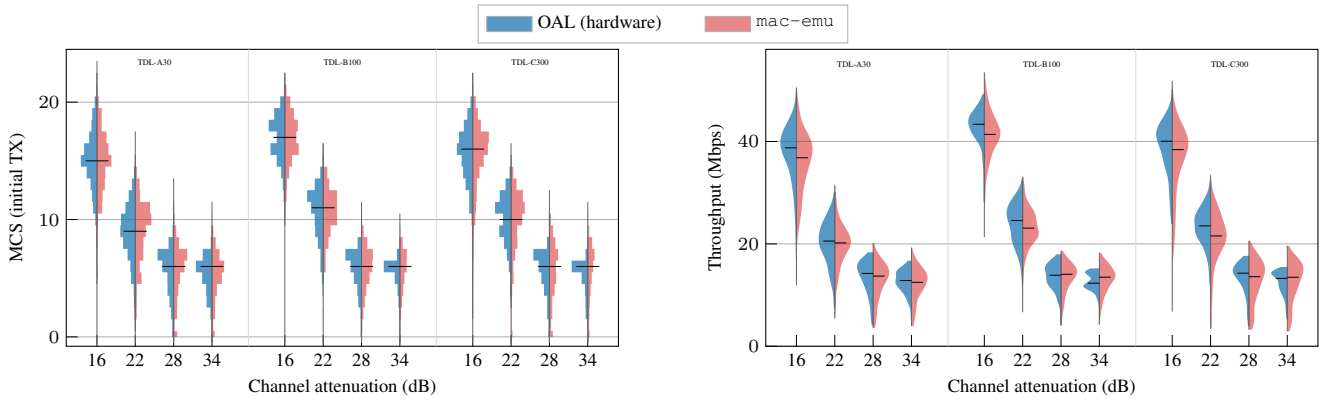
Besides the ACK/NACK results, `mac-emu` also synthesizes the measurements consumed by the scheduler. The wideband Channel Quality Information (CQI) follows the standard reporting rule: the report carries the highest CQI index whose predicted BLER, evaluated through Eq. (3), does not exceed the 10% target (TS 38.214, Clause 5.2.2.1) [26]. The scheduler also receives the per-PRB Sounding Reference Signal (SRS) channel matrix. On a real gNB this matrix comes from a channel estimator (e.g., least squares in OAI) and therefore carries estimation error. To model this error, we measure it offline with the SRS receiver in isolation (OAI's `nr_srssim`) and inject noise of matching variance into the synthesized matrix, which is derived from the current channel state; the calibrated synthesis matches the simulator with an Root Mean Squared Error (RMSE) of 0.36 dB. The synthesized CQI and SRS measurements follow the purple path in Fig. 2, where MAC-Gyver exposes them to dApps through the candidate structure. Section V uses the per-PRB SRS measurements for frequency-selective allocation.

D. Validation

We validate the two properties required for using `mac-emu` as MAC-Gyver's controlled execution environment: whether the scheduler behaves the same with `mac-emu` as with a hardware link, and whether it sustains real-time execution under cell-scale contention. The preceding subsection used the OAL setup to estimate the error-floor and PUCCH parameters. We now use the same physical testbed to compare the closed-loop MCS, throughput, and BLER observed in `mac-emu` and on hardware for the default OAI scheduler. We then verify whether the emulator sustains its real-time budget at scale. All hardware comparisons use a single-antenna link.

We compare the link-level KPIs of a single-UE deployment on `mac-emu` and on the hardware testbed under saturated DL User Datagram Protocol (UDP) traffic, across the three TDL profiles of Sec. III-C at four channel attenuations each (16, 22, 28, and 34 dB). The emulated gNB mirrors the hardware configuration, which is checked automatically at startup. Notably, the testbed reports no CQI (Channel State Information - Reference Signal (CSI-RS) is disabled), so DL link adaptation is purely BLER-driven on both platforms. Since the hardware testbed's absolute attenuation-to-SNR mapping is not recoverable exactly (the channel emulator applies a programmable attenuation between transmitter and receiver rather than an explicit path-loss model, so the absolute link budget depends on unmeasured gains along the chain), we calibrate `mac-emu`'s path loss per operating point, rerunning the experiment to find the closest MCS-distribution match with the hardware setup (via the Wasserstein distance). This enables us to evaluate how closely `mac-emu` reproduces the hardware setup despite this limitation. For each operating point, we run the experiment five times at the best-matching path loss, on both platforms.

Figure 5 reports the MCS and throughput distributions, and Fig. 6 the per-MCS initial-transmission BLER. Across the 12 points, the median Wasserstein distance between the pooled MCS distributions is 0.35 MCS steps (worst 0.94), the mean-MCS error stays below 0.35 everywhere, and median throughput agrees to -4% (all points within 13%, which is similar to



(a) MCS of initial transmissions.

(b) DL throughput.

Fig. 5: Closed-loop link adaptation in `mac-emu` and on OAL, one UE, saturated DL traffic.

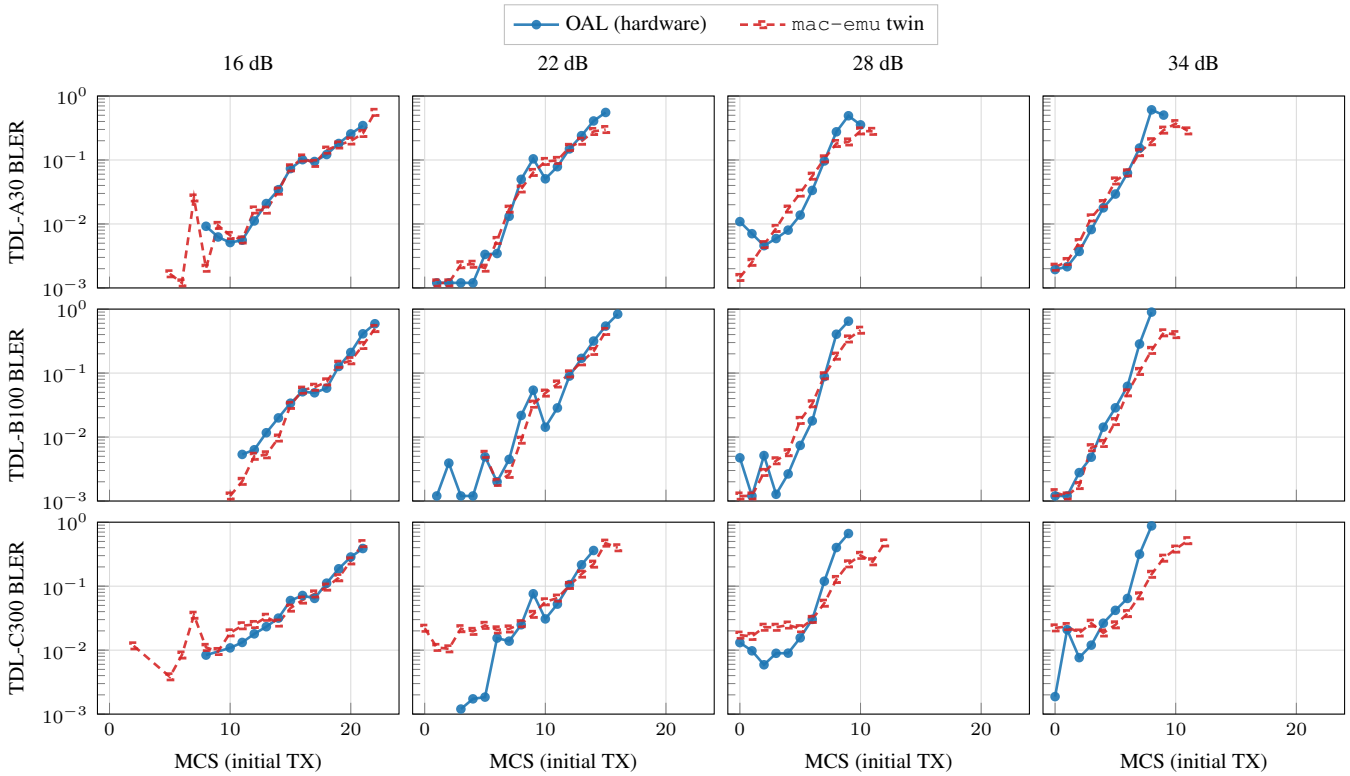


Fig. 6: Initial-transmission BLER versus MCS for the same runs as Fig. 5.

the variability we observe between runs of the same experiment on hardware) We attribute the consistently lower throughput of `mac-emu` to the PRBs number mismatch (106 (40 MHz) on `mac-emu`, 273 (100 MHz) on OAL): while we limit OAL to 106 PRBs in the scheduler, the Synchronization Signal Block (SSB) is scheduled after those PRBs. This is not the case for `mac-emu`, which loses $\approx 4\%$ of schedulable capacity (1 SSB every 20 ms, which caps the number of contiguous PRBs to 40 for that slot in our setup).

The BLER-versus-MCS curves closely follow each other through the band the outer loop targets. The remaining BLER mismatch concentrates on TDL-C300 at low MCS, where missed PUCCH-driven errors dominate: the model injects detector misses independently of the instantaneous channel state, yielding a flat BLER contribution across the low MCSs (absent on the other profiles, whose rates of missed PUCCH are negligible), whereas the hardware's low-MCS BLER retains more structure. Because the link-adaptation loop rarely selects these low MCS values in steady state, this discrepancy has no visible effect on the resulting MCS and throughput distributions.

For the scalability evaluation, we focus on verifying that `mac-emu` maintains the real-time guarantees at scale. To do this, we gradually increase the number of UEs attached to a 40 MHz SISO cell with very good channel conditions (all UEs reach MCS 28), and saturate both the UL and DL using mgen [27]. We evaluate up to 90 UEs, the maximum supported by OAI in this configuration because of its current PUCCH-resource allocation. For this experiment, the gNB thread runs on

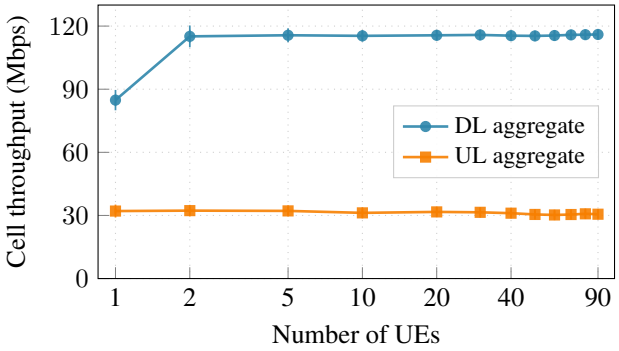
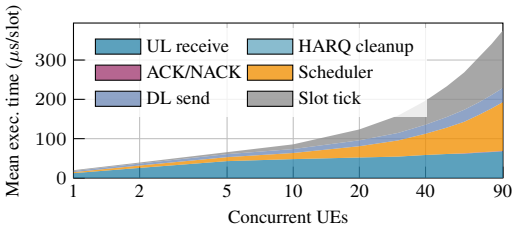
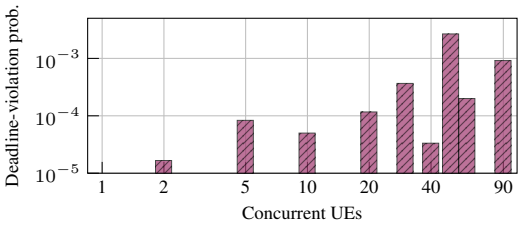


Fig. 7: Aggregate cell throughput versus concurrent UE count.



(a) Slot-thread execution time.



(b) Deadline-violation probability.

Fig. 8: `mac-emu` slot-thread scaling under saturated bidirectional traffic, versus concurrent UE count.

one isolated core, while the UE processes are distributed across ten additional cores (the whole workload runs on an NVIDIA DGX Spark with 20 CPUs, out of which 10 are performance cores, which we use for `mac-emu`). We report the aggregate throughput in Fig. 7, where we observe that `mac-emu` sustains the maximum capacity of the cell in both directions (115 and 32 $\pm$0.3 Mbps). In particular, `mac-emu` shows no computational saturation at the 90-UE limit. The only case where we reach a lower throughput is with a single UE. This is caused by a separate OAI limitation: its rigid PUCCH-allocation policy limits the maximum number of simultaneous HARQ processes in our configuration.

We also collect a breakdown of the runtime per slot in this experiment, which is reported in Fig. 8. We observe that as the number of UEs increases, the two main contributors to the runtime are the execution of the scheduler and the distribution of the slot ticks to each UE, which exhibit clear non-linear scaling. Despite this, the average runtime always stays below the critical value of 500 μs, i.e., the slot duration. The deadline violation probability is very low (around 10^{-3} at most), meaning that we near-perfectly respect the real-time deadline.

These results establish `mac-emu` as the controlled execution component of MAC-Gyver. Its channel model reproduces the

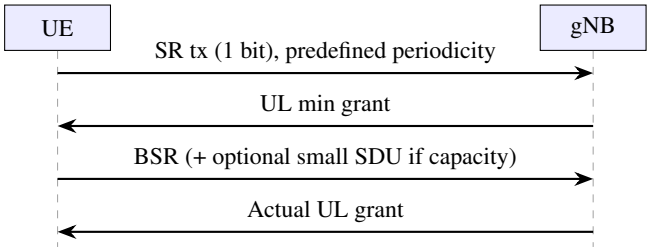


Fig. 9: Scheduling request procedure.

observations and transmission outcomes that enter the scheduler loop, while its real-time architecture sustains the complete gNB L2 stack at cell scale. The following sections use the resulting framework in two complementary ways: proactive scheduling combines persistent state and event-driven dApps in an OTA deployment, whereas frequency-selective scheduling composes per-slot decision stages and uses `mac-emu` to evaluate their interactions under controlled channel conditions.

IV. Use Case: Proactive URLLC Scheduling

Our first use case uses MAC-Gyver to implement a closed-loop proactive UL scheduler. The dApp predicts when a UE will have data and schedules its UL grant before the UE sends an Scheduling Request (SR). It adds the UE to the scheduler's candidate set while leaving resource allocation and grant validation unchanged. This use case exposes the trade-off between access latency and unused proactive grants.

A. Motivation

Our main motivation for proactive scheduling is the SR procedure, which is a major source of UL latency in 5G systems. As shown in Fig. 9, when it needs to transmit in the UL, a UE first waits for its periodic SR opportunity on the PUCCH, at which point it transmits a SR bit, indicating to the gNB it needs to transmit data. It then receives a small grant through DCI, returns a Buffer Status Report (BSR), containing the size of the payload. This enables the gNB to then schedule UL in further slots. The initial wait depends on the configured SR periodicity, typically 10–40 ms and up to 640 ms. Limited PUCCH resources require longer periodicity as the number of UEs grows. Consequently, the full procedure adds 15–50 ms and increases jitter, compared with URLLC targets of 1–10 ms [28, 29].

Note that the 5G standard provides Configured Grants (CGs) to avoid the SR delay, in which the gNB pre-configures periodic grants at fixed periodicity and offsets via RRC [2, 30]. However, CGs assume the traffic is periodic and its characteristics are known in advance, which does not hold for event-driven flows, multi-periodic traffic (e.g., a robot with multiple sensors) and more generally, unknown sparse traffic. Our approach targets these broader arrival patterns by learning from events visible by the scheduler, without requiring RRC reconfiguration.

To quantify how much latency the SR procedure adds, we compare the default scheduler of OAI with a simple always-on policy that grants the UE in every slot without waiting for an SR. Figure 10 evaluates both policies over the air with a single UE performing a 'ping' every 100 ms. The default

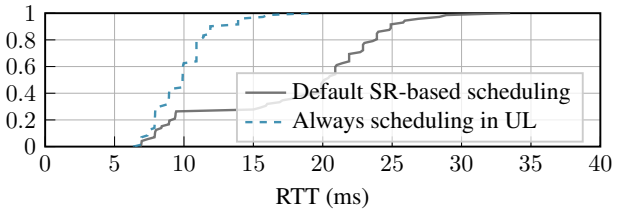


Fig. 10: RTT with and without always-on UL scheduling.

scheduler produces an RTT of 15–25 ms with a median near 20 ms. Always-on scheduling reduces this range to 7–12 ms and removes most of the variation, confirming that the SR accounts for roughly half of the observed latency. This however comes at a spectrum efficiency cost, as most grants carry no data under such sparse traffic, and the cost grows linearly with the number of URLLC UEs. We therefore use always-on scheduling as the minimum-latency reference and seek a policy that approaches its latency with fewer unused grants.

B. Problem Formulation

We formalize proactive scheduling as a multi-objective optimization over UL scheduling strategies. Within a URLLC slice of UEs $\mathcal{N}$ with homogeneous QoS requirements, we seek scheduling strategies that balance three objectives: worst-case expected latency, worst-case jitter, and total slice resource consumption. The resource-consumption ratio for UE i is

$$U_i(S) \triangleq \frac{\mathbb{E}[R_i(S)]}{R_{slice}/|\mathcal{N}|}. \tag{8}$$

The resulting problem is

$$\underset{S\in\mathcal{S}}{\text{minimize}} \quad \begin{bmatrix} \max_{i\in\mathcal{N}} \mathbb{E}[L^i_{sched}(S)] \\ \max_{i\in\mathcal{N}} V(L^i_{sched}(S)) \\ \sum_{i\in\mathcal{N}} U_i(S) \end{bmatrix}. \tag{P-URLLC}$$

where $\mathcal{N}$ is the set of the $|\mathcal{N}|$ UEs in a URLLC slice, assumed to share homogeneous QoS requirements. $\mathcal{S}$ is the set of feasible scheduling strategies, with $S \in \mathcal{S}$ being one such strategy. For each UE $i \in \mathcal{N}$, $L^i_{sched}(S)$ is the latency induced by S, $V(L^i_{sched}(S))$ is the latency variance (jitter), $U_i(S)$ is the resource-consumption ratio, $R_i(S)$ is the random variable representing the instantaneous resources allocated to i, and R_{slice} is the total PRB budget reserved for the URLLC slice.

The vector objective in Eq. P-URLLC is minimized in the Pareto sense where a feasible strategy is Pareto-optimal if no other strategy achieves values no larger in all three objectives and a strictly smaller value in at least one. Solving Eq. (P-URLLC) requires the distribution of scheduling latency, which depends on the unknown UL arrival process. We therefore use an online policy that estimates this process from events available inside the MAC scheduler. Two parameters, the tolerance window TW and slot restriction N, control how closely proactive grants follow each prediction and set the policy's position on the latency-overhead trade-off. A state machine alternates between collecting unbiased arrival samples and scheduling from the resulting predictions.

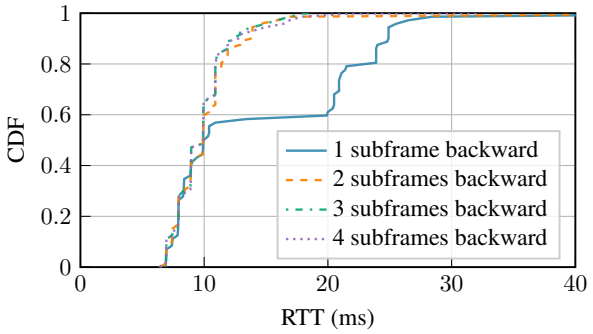


Fig. 11: RTT with a DL-aware proactive scheduler at varying lookback windows.

C. Architecture and State Machine

We first evaluate a simple prediction rule before presenting the full architecture. For Fig. 11, we implement a scheduling dApp that issues a proactive UL grant whenever the scheduler has transmitted DL data to the UE within the preceding one to four TDD periods. The figure reports RTT for each lookback window. Extending the window increases the number of proactive grants linearly. A two-period lookback reduces the median RTT to approximately 9 ms, close to always-on scheduling. Unlike always-on scheduling, this rule issues grants only during the two periods following each DL transmission.

We now generalize this basic principle beyond a fixed lookback window. As shown in Fig. 12, our solution trains a per-UE learner which predicts the next UL arrival from MAC events, and a state machine determines when to use the prediction for proactive scheduling.

We implement the proactive scheduler as a combination of three dApps using the interfaces shown in Fig. 1: candidate request (A), DL transmission observation (B), and UL observation (C). The DL observer dApp reports each constructed transport block, while the UL observer dApp reports the outcome of each UL grant, including its decoding outcome, received payload, padding, grant parameters, and whether the candidate was requested by a dApp. A received payload and an unused proactive grant are therefore two outcomes of the same typed event reported by the UL observer dApp (C). Together, these observations update the per-UE predictor maintained in the dApps' shared persistent context. During candidate collection, the candidate-request dApp is invoked once for each structurally eligible UE. When the online learner predicts that a proactive grant is due, this dApp requests that the UE enter the candidate set. Because no BSR is available before the SR, the buffer state remains unknown. In our configuration, the independently selected PRB-allocation dApp therefore assigns the minimum grant, after which the candidate follows the standard allocation and validation path. The architecture accepts any arrival predictor that outputs a predicted slot. We evaluate an Exponentially Weighted Moving Average (EWMA) and linear regression in this work. More expressive models, including Neural Networks (NNs) and gradient-boosted trees, can use the same interface; we leave their evaluation to future work.

State machine. A state-machine instance is associated with each group of UEs that shares an arrival distribution. When no

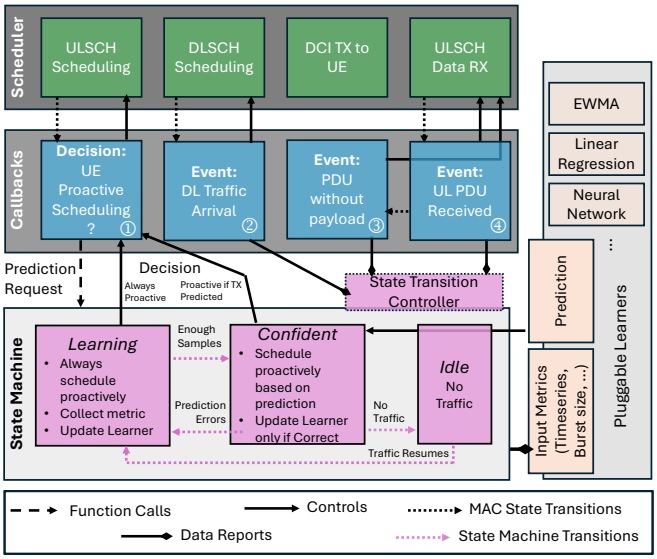


Fig. 12: Proactive scheduler architecture.

such prior is available, each UE receives its own instance. A group may share one instance when its members are known to have the same statistics, such as identical sensors on a fixed reporting schedule. We describe one instance below. The scheduler runs all instances independently.

At initialization, the instance is in the *Learning* state, where it schedules proactively in every slot. This both masks the SR delay and collects an unbiased dataset of true UL arrival times. In parallel, the learner is updated from any subset of scheduler-observable features, including past UL inter-arrival times, DL event sizes and timestamps, DL-to-UL response times, and traffic class. The learner emits a predicted slot offset for the next UL arrival. The instance enters *Confident* once a learner-specific readiness condition is met, such as a minimum sample count or a bound on prediction variance.

In the *Confident* state, scheduling follows the learner's prediction through two operator-tunable parameters. The tolerance window TW admits scheduling within $\pm$TW TDD periods of the prediction. The slot restriction N limits proactive grants to the last N slots of each TDD period. Smaller (TW, N) reduce overhead by committing more closely to the predicted slot. Larger values widen the safety window at the cost of additional grants. In the language of Eq. (P-URLLC), *Learning* realizes the minimum-latency extreme at maximum resource use, while $(\mathrm{TW}, N) = (0, 1)$ approaches the minimum-overhead prediction-driven configuration. Intermediate values trace the empirical latency-overhead frontier. The achievable trade-off depends on prediction accuracy.

Two online counters determine whether the instance remains *Confident*. Consecutive reactive transmissions count missed predictions, while unused proactive grants determine the false-positive rate. Crossing a configured threshold, such as three consecutive misses or a 10% false-positive rate, returns the instance to *Learning*. A learner may also emit (TW, N) dynamically. This lets it absorb drift by temporarily widening the safety window without returning fully to *Learning*. A third *Idle* state pauses prediction after extended inactivity, such as 100 slots without UL traffic. When traffic resumes, the instance returns to *Learning*.

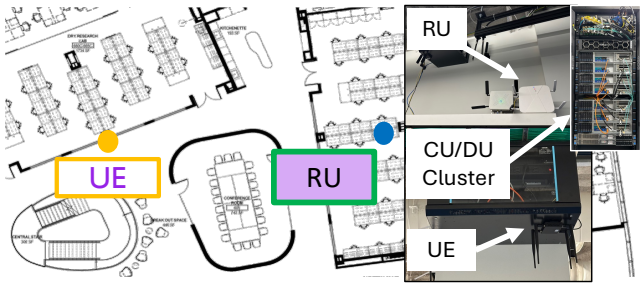


Fig. 13: Over-the-air testbed for proactive-scheduler evaluation.

Together, the three states balance latency, jitter, and overhead under unknown or drifting traffic without RRC reconfiguration or prior knowledge of the arrival process.

D. Evaluation

To exercise MAC-Gyver in an OTA deployment, we evaluate the proactive scheduler on the testbed shown in Fig. 13. The deployment is built on the X5G testbed [17] and the AutoRAN framework [31]. The OAI gNB runs on a Dell PowerEdge R760 server connected to a Foxconn RPQN RU operating in band n78 with 100 MHz bandwidth and numerology $\mu = 1$. A Sierra Wireless EM9293 UE communicates over the air under fair channel conditions, with a median CQI of 13. The network uses the DDDDDDDSUU TDD pattern with a 10-slot period.

We design three complementary scenarios in which proactive scheduling can leverage different forms of temporal structure.

Request-response. We generate ICMP echo requests with a 300-byte payload from the User Plane Function (UPF) to the UE at controlled intervals. This scenario represents query-response applications such as industrial sensors responding to polling or roadside infrastructure querying vehicle status. The predictable DL-to-UL pattern allows us to evaluate RTT-based learning, where the scheduler predicts UL responses based on prior DL transmissions. We configure the scheduling dApp with an EWMA learner.

Size-dependent edge inference. We emulate workloads in which a central node queries an edge device, which runs local ML inference and returns the result. Both the inference latency and the resulting UL payload may depend on the incoming DL traffic. We instantiate this scenario by running an Arctic Embed XS text-embedding model on the UE. It receives a variable-length prompt over DL, performs inference whose duration depends on the input length, and returns the embedding over UL. The server at the UE and the client at the UPF communicate over UDP. We vary the input using random word sequences of different lengths, which produce different burst sizes at the MAC layer. We configure the scheduling dApp to use linear regression on the DL burst size.

Periodic autonomous reporting. We generate periodic UL-only traffic at fixed intervals between 10 and 100 ms, with no preceding DL trigger. This represents workloads such as heartbeat messages and autonomous sensor reporting. Since there is no DL correlation to leverage, the scheduler must learn the inter-arrival pattern. We use MGEN to send 97 packets per second. We select this prime-number rate so that the position of packets within the TDD period does not repeat. It produces

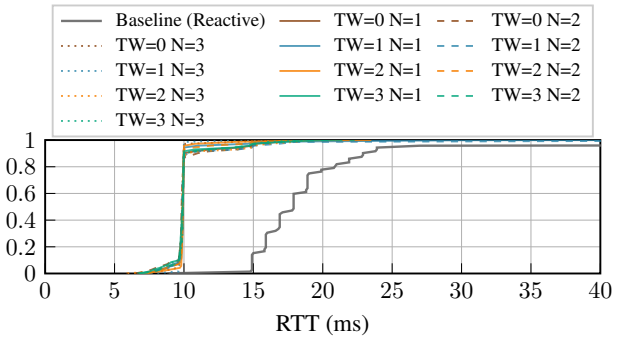


Fig. 14: RTT for the request-response scenario.

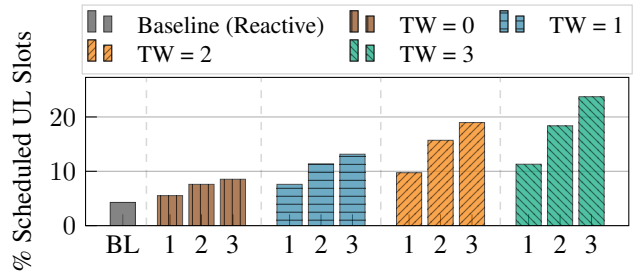


Fig. 15: Request-response overhead; bottom labels show N.

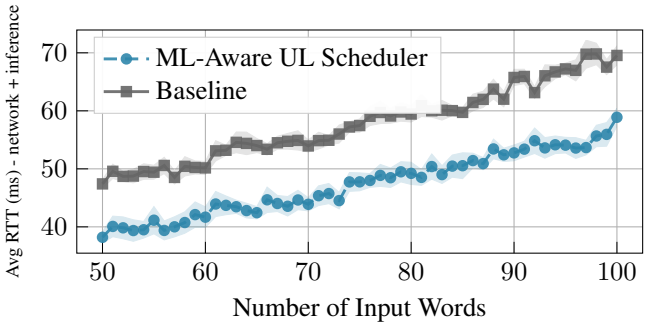


Fig. 16: End-to-end latency for the size-dependent edge inference scenario.

a 10.31 ms inter-arrival time, between the standardized CG periodicities of 10 and 20 ms. We configure the scheduling dApp with an EWMA learner.

For evaluation, we disable the Confident-to-Learning fallback transition that would normally be triggered by accumulated prediction errors. Otherwise, the measured latency and overhead would mix the predictor's accuracy in the Confident state with the always-schedule safety net of the Learning state, which by construction occupies the 100%-overhead, minimum-delay corner. Disabling the fallback isolates the first effect, so the reported results reflect Confident-state prediction quality alone. Our baseline is the default OAI scheduler, which does not perform proactive scheduling.

For each scenario, we evaluate the latency CDFs across the (TW, N) sweep. For the request-response scenario, we also evaluate the overhead, defined as the fraction of UL slots in which the UE is scheduled. The sweep traces an empirical latency-overhead trade-off. A sharp knee occurs when one (TW, N) configuration achieves near-minimum overhead and near-minimum latency, while more aggressive configurations provide no further latency reduction. The location of this knee also indicates prediction quality. For example, a knee at $(\mathrm{TW}, N) = (0, 1)$ means that the learner localizes UL arrivals to one TDD period and one eligible UL slot.

Request-response results. We vary TW between 0, which schedules proactively only when the TDD period matches the prediction exactly, and 3, which covers ± 3 periods around the prediction. We vary N between 1 and 3 because the configured pattern has at most three UL transmission opportunities per TDD period, comprising two U slots and one mixed S slot.

Figure 14 presents the latency results. For every (TW, N) configuration, the distribution improves substantially over the baseline. The median drops from approximately 20 ms with a long tail to 10 ms. The 90th percentile is also 10 ms, and the long tail disappears. As shown in Fig. 15, the overhead, measured as the fraction of UL slots with some resources scheduled either proactively or reactively, scales linearly with TW and N. The optimal configuration for this scenario is $\mathrm{TW} = 0$ and $N = 1$, where the overhead is 7%. The baseline has the lowest overhead, with 4% of slots scheduled. We present detailed overhead results only for request-response. Overhead scales similarly with TW and N in the other scenarios, with absolute values proportional to their traffic rates.

Size-dependent edge inference results. We use $\mathrm{TW} = 2$ and $N = 1$. As shown in Fig. 16, the scheduler predicts the UL arrival time from the DL burst size, reducing end-to-end delay by approximately 10 ms over the baseline across all input sizes. This is consistent with the network latency saved in the request-response scenario.

Periodic autonomous reporting results. Figure 17 presents the results for the third scenario. We vary TW between 0 and 3 and N between 1 and 3. A tolerance window of $\mathrm{TW} = 0$ already improves the CDF over the baseline, but the best results appear from $\mathrm{TW} = 1$, with a median one-way latency of 7 ms and a 99th percentile of 10 ms. The shift in the optimal TW reflects the traffic pattern. The 97-packet/s rate produces a 10.31 ms inter-arrival time that is not a multiple of the TDD period, so successive packets move across slot boundaries within the pattern. A predictor with $\mathrm{TW} = 0$ commits to one TDD period and misses the arrival as drift accumulates. A tolerance of $\mathrm{TW} = 1$ absorbs the drift at low overhead. A CG cannot apply the same optimization because its fixed periodicities of 10 and 20 ms lock the grant to one phase. The same drift would steadily desynchronize the CG schedule from the actual arrivals. The difference between $N = 1, 2, 3$ is at most 0.1 ms. For this regular traffic, scheduling a few slots earlier has only a small effect on UL latency. The optimal operating point is therefore traffic-dependent, and the architecture allows the learner to emit (TW, N) from the observed statistics.

Overall, this use case demonstrates how MAC-Gyver composes observations, persistent state, and candidate requests across scheduling opportunities. The DL and UL observer dApps update the predictor, while the candidate-request dApp acts on its output; resource allocation and grant validation remain unchanged. The resulting policy executes inside the OAI scheduler over the air. We next exercise the complementary set of interfaces: per-slot decision stages evaluated under controlled channel conditions with `mac-emu`.

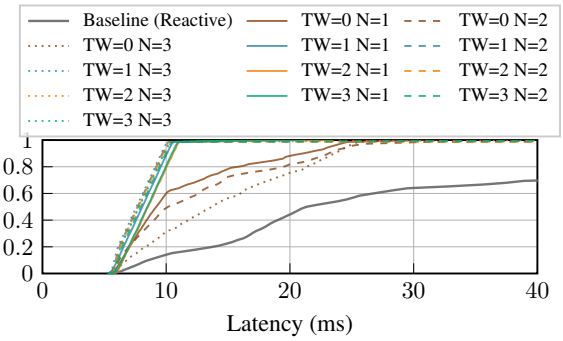


Fig. 17: One-way UL latency for the periodic autonomous reporting scenario.

V. Use Case: Frequency-Selective UL Scheduling

Our second use case uses MAC-Gyver to implement a frequency-selective UL scheduler. The scheduler leverages frequency-domain channel variations to assign each UE a favorable contiguous sub-band. We show how MAC-Gyver enables us to leverage SRS pilots for resource allocation, and how `mac-emu` enables evaluation across large-scale mobility and power-limited scenarios.

A. Motivation and Problem

The potential gain from frequency-selective scheduling comes from multi-user frequency diversity, i.e., multipath propagation produces different strong and weak frequency regions for different UEs. This creates the opportunity of matching each grant to a favorable region, thereby increasing aggregate throughput within the same spectrum budget. To realize this gain, channel estimates need to be fresh. Indeed, the frequency response estimate (measured through SRS in the UL or through CQI in the DL) that the gNB observes must be close enough to the current channel state. As Doppler increases, the channel deviates from the pilot estimates faster, and the real ordering of candidate sub-bands may change compared to the stale information available at the gNB [32]. In the UL, the available gain may also depend on the UE's operating point (i.e., its power budget and the state of the power-control loop) [33].

These considerations have motivated a broad literature on frequency-selective resource allocation, primarily on the DL. Early multi-user OFDM schedulers jointly assign individual subcarriers and select their transmission rates [34, 35]. These formulations do not map directly to 5G, where scheduling operates at PRB granularity, one MCS applies across a transport block, and UL allocations are subject to standardized resource-allocation and waveform constraints. More recent designs use quantized sub-band CQI reported by the UE. For example, measurements on commercial LTE hardware characterize the gain from sub-band CQI reporting [32], while SARA uses sub-band Rank Indicator (RI), Precoding Matrix Indicators (PMI), and CQI reports in 5G-LENA [36]. Depending on the configured resource-allocation type, the DL scheduler may also distribute an allocation across multiple resource-block groups.

A smaller body of work studies frequency-domain scheduling on the UL. Earlier work primarily formulates the allocation problem under the contiguity constraint imposed by the UL waveform. Krishnamoorthy *et al.* propose frequency-semi-selective scheduling to reduce channel-reporting overhead for DFT-S-OFDM [37], while Calabrese *et al.* construct contiguous channel-aware allocations using a search tree [38]. Lee *et al.* further show that proportional-fair frequency-domain scheduling under this constraint is NP-hard [4]. Gutierrez *et al.* provide a closer connection to the LTE measurement procedure [39]. They derive a scalar effective Signal to Interference plus Noise Ratio (SINR) for each UE–PRB pair in the Vienna LTE-A simulator, use Recursive Maximum Expansion to construct contiguous allocations, and select a common MCS from the channel quality across each assigned interval. Because the allocation procedure is shared across their comparisons, the evaluation primarily measures the effect of link adaptation instead of isolating the gain from frequency-selective placement. Their formulation also does not consider how frequency-domain channel quality depends on the selected rank, precoder, or the UE's power distribution across a grant with a variable number of PRBs.

Taken together, these gaps motivate studying frequency-selective allocation within the complete scheduling loop, which we do using MAC-Gyver. We formulate and evaluate two real-time allocation frequency-selective policies. The first restricts each UE to a fixed target number of PRBs, while the second retains a bounded set of candidate PRB counts. We first evaluate these policies offline through simulation, before evaluating whether the properties observed in simulation carry within the real network stack, and how they compare again the proportional fair baseline.

B. Design and Implementation

We now formulate the frequency-selective allocation problem and derive two policies that restrict its search space sufficiently for per-slot execution. We then evaluate their expected allocation quality and computational cost offline in simulation before executing them in `mac-emu`.

Sub-band utility. Both our algorithms start from constructing a utility profile based on each UE's SRS feedback. We read the complex SRS estimates exposed through the shared candidate structure of MAC-Gyver. For each candidate UE u and PRB k, we apply the rank and precoder selected by the rank/precoder dApp (stage 2 in Fig. 1) and estimate the post-Minimum Mean Square Error (MMSE) SINR $\gamma_{u,\ell}(k)$ of every scheduled layer ℓ.

Since SRS uses a per-UE power-control loop distinct from those of other UL channels, the magnitude of the SRS estimates does not directly represent the PUSCH operating point. We therefore treat the SRS estimates as a relative frequency profile and combine them with the latest PUSCH-measured SINR. We then define the per-PRB utility as the following rate proxy:

$$q_u(k) \triangleq \sum_{\ell=1}^{L_u} \log_2(1+\gamma_{u,\ell}(k)), \tag{9}$$

where L_u is the number of layers selected for UE u. We represent a candidate grant by its first PRB s and the number

w of contiguous PRBs it contains. Its aggregate utility is

$$R_u(s,w) \triangleq \sum_{k=s}^{s+w-1} q_u(k). \tag{10}$$

Allocation problem. Let p_u denote the proportional-fair priority of UE u, B the number of available PRBs, U the number of candidate UEs, and K the number of additional grants permitted by the remaining PDCCH budget when the dApp is invoked. $\kappa \triangleq \min(U,K)$ is the maximum number of candidates that the allocation can select. Given the interval utilities above, the allocator seeks a set $\mathcal{A} = \{(u, s_u, w_u)\}$ of non-overlapping intervals that maximizes

$$\max_{\mathcal{A}} \sum_{(u,s_u,w_u)\in\mathcal{A}} p_u R_u(s_u,w_u), \tag{11}$$

subject to $|\mathcal{A}| \le \kappa$, at most one interval per UE, and the contiguity and feasibility constraints of the UL grant. An exact Dynamic Program (DP) can index its state by the next available PRB and the subset of users that remain available. For a candidate set $\mathcal{W}$ of PRB counts, with $W = |\mathcal{W}|$, this formulation requires $O(B \cdot 2^U)$ states and $O(BUW \cdot 2^U)$ time in the worst case, making DP unsuitable for real-time execution. We therefore formulate two real-time policies that restrict different dimensions of this search space.

Fixed PRB-count placement. Our first policy (FREQSEL-FIXED) assumes that up to κ candidates will be selected and divides the B available PRBs equally among them, and enforces a minimum grant size $w_{\min}$. The target number of PRBs is

$$w_{\text{tgt}} \triangleq \max\left(w_{\min}, \left\lfloor \frac{B}{\kappa} \right\rfloor\right). \tag{12}$$

For each UE u, the policy examines every contiguous run of available PRBs. Within a run of length ℓ, it evaluates all starting positions for an interval containing $w = \min(w_{\text{tgt}}, \ell)$ PRBs. Across all runs, it retains the interval (s_u, w_u) with the largest $R_u(s_u, w_u)$. This produces one candidate interval per UE with weight $p_u \times R_u(s_u, w_u)$. Then, since preferred intervals of each UE may overlap, a weighted-interval-scheduling DP selects the maximum-weight non-overlapping subset containing at most κ intervals.

Variable PRB-count placement. Our second policy, FREQSEL-REFINED, separates user ordering from PRB-count refinement. First, a maximum-weight assignment maps κ distinct users to κ ordered intervals containing equal numbers of PRBs. Let $\pi = (u_1, \ldots, u_\kappa)$ denote the resulting user order. Holding π fixed, an ordered DP chooses (s_{u_j}, w_{u_j}) such that

$$s_{u_j} + w_{u_j} \le s_{u_{j+1}}, \qquad w_{u_j} \in \mathcal{W}, \quad 1 \le j < \kappa. \tag{13}$$

The DP maximizes $\sum_{j=1}^{\kappa} p_{u_j} \times R_{u_j}(s_{u_j}, w_{u_j})$ over the feasible positions and PRB counts.

Computational cost. Both policies construct $q_u(k)$ and its cumulative values in $O(B)$ time per UE. The cumulative representation then allows an interval score to be obtained with two lookups and one subtraction. For FREQSEL-FIXED, scanning the starting positions costs $O(UB)$, constructing the interval-compatibility relation costs $O(U^2)$, and selecting at most κ intervals costs $O(U\kappa)$. Its overall complexity is therefore $O(UB + U^2 + U\kappa)$. For FREQSEL-REFINED, assigning κ intervals to U users costs $O(U\kappa^2)$, while evaluating W candidate PRB counts across B positions for each selected user costs $O(\kappa BW)$. Including utility-profile construction, its overall complexity is $O(UB + U\kappa^2 + \kappa BW)$.

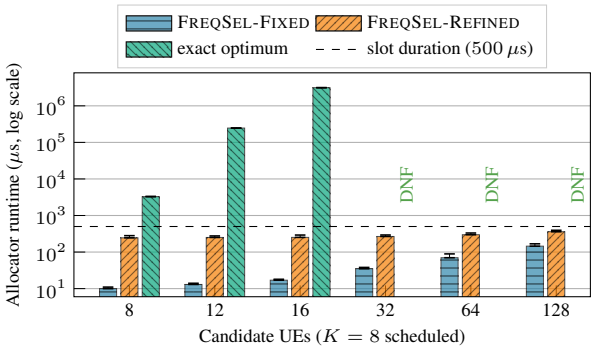


Fig. 18: Scheduler execution time (offline execution).

Offline policy evaluation. We first evaluate whether the two restricted searches can execute within the slot duration. We fix $B = 106$, $K = 8$ (and hence $\kappa = 8$), and $W = 30$, vary the number of candidate users from 8 to 128, and measure the execution time outside the scheduler loop. Figure 18 shows that both policies remain within the slot budget: with 128 candidates, their 99th-percentile execution times are 168 μs for FREQSEL-FIXED and 394 μs for FREQSEL-REFINED. In contrast, the exact solver already requires 3.3 ms for eight users and 247 ms for twelve users.

We then compare the allocation quality of the two real-time policies. Each channel instance contains eight independent static rank-2 users over a $B = 106$-PRB carrier under a CDL-B channel with 300-ns delay spread. We replay the searches performed by FREQSEL-FIXED and FREQSEL-REFINED with equal proportional-fair weights and no pre-existing allocations. For every selected interval, we derive the post-MMSE SINRs from the exact channel state and select the MCS that maximizes expected rate under `mac-emu`'s calibrated BLER model. We divide the resulting aggregate expected rate by B to obtain the expected cell spectral efficiency in bit/s/Hz.

Figure 19 reports this quantity while varying path loss. FREQSEL-REFINED outperforms FREQSEL-FIXED throughout the sweep, with 10.98 against 10.27 bit/s/Hz at 96 dB path loss, and 110 dB, they achieve 6.29 against 5.46 bit/s/Hz at 110 dB. Overall FREQSEL-REFINED's advantage grows from 7.0% to 15.2% as the users become power-limited. Taken together, Figs. 18 and 19 identify FREQSEL-REFINED as a promising real-time policy in the offline evaluation: the implementation remains within the slot budget, and it outperforms FREQSEL-FIXED.

C. MAC-Gyver-based Evaluation

We use MAC-Gyver to evaluate frequency-selective allocation inside the complete OAI scheduling loop in `mac-emu`. We first select between the two real-time variants introduced above and then compare the retained FREQSEL-FIXED policy with proportional fairness across operating conditions. The experiments use saturating UL Multi-Generator (MGEN) traffic,

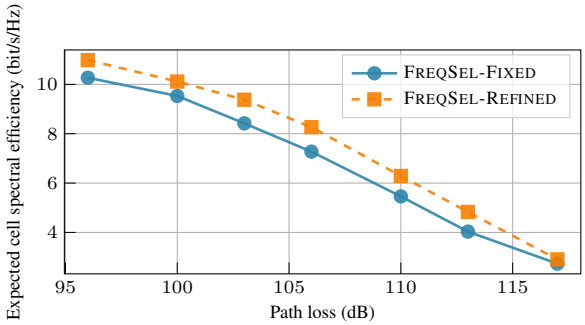


Fig. 19: Simulation-based expected cell spectral efficiency for 8 static users under CDL-B 300-ns channel.

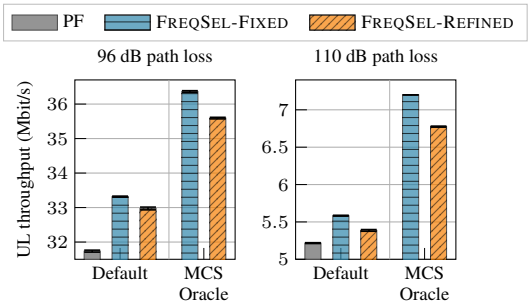


Fig. 20: Aggregate UL throughput in `mac-emu`.

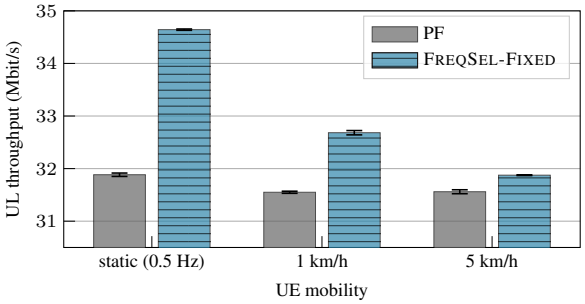


Fig. 21: Aggregate UL throughput versus user speed.

a CDL-B 300 ns channel and two-layer MIMO. Unless stated otherwise, they use 32 users and periodic SRS with an interval of 160 slots (80 ms).

Mobility models. For moving users, we set the maximum Doppler according to $f_D = vf_c/c$; at our 3.5-GHz carrier frequency, speeds of 1, 2, and 5 km/h correspond to 3.24, 6.48, and 16.20 Hz, respectively. For static UEs, we use 0.5 Hz to represent channel variations caused by moving environmental scatterers, as observed on fixed wireless links [40].

We evaluate with homogeneous scenarios (all UEs either moving or static), as well as a mix of static and moving UEs. Additionally, we also use a Lévy-walk mobility model for one scenario, in which users alternate between movement at 5 km/h and quasi-static pauses. Pause durations follow a truncated Pareto distribution with exponent 0.8 over 30–300 s, while movement durations are exponentially distributed with a mean of 60 s. During pauses, the channel continues to evolve at 0.5 Hz. This captures the alternating movement and heavy-tailed pause behavior observed in human-mobility traces [41].

Policy selection. We begin by determining whether superiority of FREQSEL-REFINED over FREQSEL-FIXED is also observed in end-to-end network performance. We execute FREQSEL-FIXED and FREQSEL-REFINED within MAC-Gyver as PRB allocation dApps. The experiment contains 16 quasi-static users, eight users at 2 km/h, and eight users at 5 km/h. Figure 20 compares aggregate UL throughput at 96 and 110 dB path loss. In each panel, the left group compares proportional fairness and our two policies. At 96 dB, FREQSEL-FIXED and FREQSEL-REFINED achieve 33.32 and 32.97 Mbit/s, corresponding to gains of 5.0% and 3.9% over proportional fairness. At 110 dB, they achieve 5.58 and 5.39 Mbit/s, corresponding to gains of 7.0% and 3.3%. Thus, FREQSEL-REFINED remains below FREQSEL-FIXED, and the improvement suggested by the offline optimization does not carry when evaluating with the full scheduler.

The oracle-MCS bars on the right of each panel test whether this reversal is caused by the interaction with link adaptation. We replace the default MCS-selection dApp with an oracle dApp that uses the exact `mac-emu` channel state to select the highest MCS whose predicted BLER does not exceed 13%. At 96 dB, FREQSEL-FIXED and FREQSEL-REFINED achieve 36.36 and 35.59 Mbit/s, respectively. At 110 dB, they achieve 7.20 and 6.78 Mbit/s. Although the oracle increases throughput for both policies, it does not change their ordering: FREQSEL-REFINED remains 2.1% and 5.9% below FREQSEL-FIXED at the two operating points.

This experiment illustrates how MAC-Gyver supports policy shaping within the complete system rather than from an isolated optimization objective. By replacing the allocation and link-adaptation stages independently while retaining the rest of the scheduling loop, we find that refining the number of allocated PRBs increases computational and implementation complexity without improving throughput. We therefore retain FREQSEL-FIXED for the remaining experiments, which compare it with proportional fairness under the common CDL-B 300-ns channel at 96 dB path loss, unless stated otherwise.

Mobility Experiments. We first isolate the effect of channel evolution by placing all 32 users in the same mobility state. Figure 21 compares the quasi-static, 1-km/h, and 5-km/h cases. The gain of FREQSEL-FIXED is largest in the quasi-static cell and decreases with mobility as the channel evolves between the SRS observation and the resulting grant. This trend is consistent with the existing studies (e.g., most gain lost at 10 km/h with 20 ms channel sounding intervals [32]).

Mixed mobility. Next, we evaluate FREQSEL-FIXED in cells with mixed mobility. In Figure 22, we vary the number of 5-km/h users in a 32-user cell, with the remaining users following the quasi-static model. The gain over proportional fairness decreases as the mobile fraction increases, from 7%, or 2.0 Mbit/s, when every user is quasi-static to no measurable gain when every user is mobile. We then show in Figure 23 how this gain is distributed in a cell containing 16 quasi-static and 16 mobile users. We observe that frequency-selective allocation shifts the throughput distribution of the quasi-static users toward higher rates, while the distribution of the mobile users remains close to that obtained with proportional fairness, as channel evolution

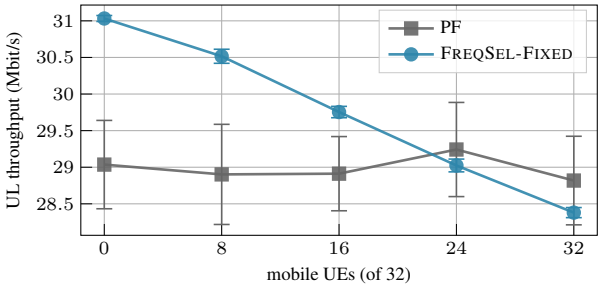


Fig. 22: Aggregate UL throughput.

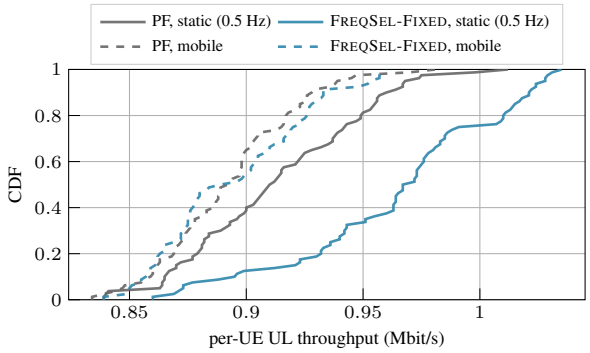


Fig. 23: Per-user UL throughput in a cell with 16 quasi-static users and 16 users moving at 5 km/h.

and SRS aging removes most of the benefit of channel-aware PRB allocation.

Lévy-walk mobility. The preceding experiments keep each user in one mobility state throughout the run. We finally examine whether the same behavior holds when individual users alternate between moving and paused periods according to the Lévy-walk model. Figure 24 reports the per-grant MCS distribution conditioned on the user's state. During quasi-static pauses, FreqSel-Fixed shifts the distribution toward higher MCS values relative to proportional fairness. During movement at 5 km/h, the two distributions largely overlap. The benefit therefore follows the user's current mobility state: frequency-selective placement improves link quality while the observed frequency profile remains sufficiently stable, but provides little value after that profile decorrelates.

Overall, FreqSel-Fixed improves aggregate UL throughput by up to 7% in the complete scheduler, with gains concentrated among users whose channels vary slowly. More importantly, this set of experiments illustrates the value of exposing scheduler decisions as separate dApp stages. The offline evaluation favors FreqSel-Refined, but the policy ordering reverses when both algorithms execute inside the complete OAI scheduling loop. Because resource allocation and MCS selection are decoupled, we can replace the default link adaptation with an oracle dApp while leaving the algorithms and the rest of the scheduler unchanged. Even under these idealized conditions, the reversal persists. Combined with `mac-emu`'s controlled mobility and power conditions, this decomposition lets us identify which the best algorithm within the complete system.

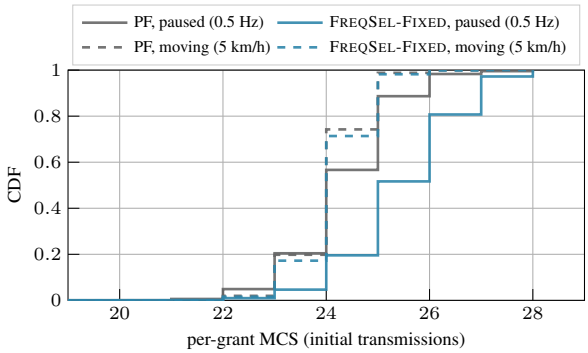


Fig. 24: Per-grant MCS conditioned on UE state (Lévy-walk).

VI. Related Work

We compare MAC-Gyver with existing experimentation platforms and with prior work on the two scheduling problems.

A. Experimentation Frameworks, Platforms, and Emulation

Open RAN stacks such as OAI [42] and srsRAN/OCUDU [43] implement a full protocol stack compliant with 3GPP NR Release 15/16. Their per-slot schedulers — of which the internals are not standardized, leaving the vendor room to develop their own logic — however, remain monolithic and represent the motivation for the introduction of MAC-Gyver. To our knowledge, MAC-Gyver is the first framework to expose the in-MAC per-slot decision as a pluggable interface in a production RAN stack.

External controllers provide a complementary control tier. The Open RAN (O-RAN) architecture hosts control logic in near-Real-Time (RT) and non-RT RAN Intelligent Controllers (RICs) [44]; FlexRIC connects these controllers to OAI through an E2 agent and service-oriented Software Development Kit (SDK) [45]; and ColO-RAN develops and evaluates machine-learning xApps on programmable platforms [7]. These controllers steer UE-level policies over tens of milliseconds or longer, and this is not compatible with real-time scheduling decisions. A MAC-Gyver dApp applies such a policy directly inside the per-slot scheduler. This design follows the dApp model, which places sub-millisecond inference and control inside the gNB [8], and its E3 interface for attaching dApps and consuming real-time data [10]. MAC-Gyver specializes this model into synchronous scheduling stages and additionally lets stateful dApps observe transmission outcomes and orchestrate controls across scheduling opportunities.

Existing evaluation platforms trade hardware fidelity for scale and accessibility. Colosseum connects Software-defined Radios (SDRs) through an Field Programmable Gate Array (FPGA)-based RF channel emulator [18], while POWDER provides a city-scale OTA testbed [46]. Both retain a real PHY and radio hardware, which require shared facilities and limit the number of simultaneous UEs. ns-3 [47] and 5G-LENA [48] scale through simulated time and reimplemented schedulers. Sionna provides differentiable link-level models [49], and NVIDIA Aerial implements a Compute Unified Device Architecture (CUDA)-accelerated PHY [50].

Closer to `mac-emu`, 5G-EMANE proposes a multi-user proxy over nFAPI for LTE and NR OAI, providing a PHY-less Layer 2 proxy [51]. Its 5G evaluation demonstrates up to 15 users. However, the authors report approximately 30 Mbit/s with 40% packet loss in DL. Moreover, its public implementation[3] has seen no commits since January 2024.

B. Predictive and Proactive URLLC Scheduling

New UL traffic can wait for a periodic SR, a small grant, and a BSR before the gNB issues its data grant. The TDD pattern adds slot-alignment delay, and commercial 5th generation (5G) standalone networks report median UL round-trip times of 50–70 ms [52]. Configured grants remove the SR wait by pre-allocating periodic UL resources [2, 30]. Their fixed period and phase serve strictly periodic traffic. Event-driven and multi-source arrivals can waste resources or miss reserved opportunities. Configured grants also require the application and RRC configuration at the UE to remain synchronized. Furthermore, a traffic period that is not aligned with the TDD pattern drifts relative to the available UL slots.

Learning-based schedulers generally act after demand becomes known. LEASCH uses deep reinforcement learning to allocate resource blocks at the MAC layer [5]. Other work jointly allocates resources to URLLC and Enhanced Mobile Broadband (eMBB) traffic [3], controls that split through an O-RAN xApp [53], or evaluates data-driven control at scale with ColO-RAN [7]. Our scheduler addresses the preceding access delay. It learns arrivals online from in-RAN events and issues a grant before the UE sends an SR. The Layer 2 dApp needs no offline dataset or application-to-RRC synchronization. To our knowledge, prior work has not demonstrated online arrival prediction and proactive UL grants in a production RAN stack.

C. Frequency-Selective and Channel-Aware Scheduling

Frequency-selective scheduling first emerged in multi-user OFDM. Wong *et al.* [34] minimize DL power under per-user rate constraints, while Rhee and Cioffi [35] maximize capacity through dynamic subchannel assignment. Later DL work measured a gain of roughly two CQI steps in Long Term Evolution (LTE) field trials [32] and used per-sub-band RI, PMI, and CQI in the SARA scheduler for 5G-LENA [36].

The UL adds a contiguity constraint because SC-FDMA (DFT-S-OFDM) assigns each UE a contiguous set of PRBs. This restriction prevents the scheduler from assigning an arbitrary set of strong sub-bands to one UE. Lee *et al.* [4] prove that proportional-fair frequency-domain scheduling is NP-hard under this constraint. Consequently, prior UL schedulers use heuristics such as candidate-tree search [38] or a coarse sub-band grid [37], primarily evaluated through analysis and link- or system-level simulation.

Our allocator extends this UL work in three ways. First, it consumes per-PRB SRS observations inside OAI and places each user's contiguous grant on a favorable sub-band. Second, it derives real-time policies with different restrictions on the exact contiguous-allocation problem. Finally, it uses offline analysis to shape these policies and integrated execution to select between them. To our knowledge, prior work has not combined these properties in a production RAN stack.

[3]https://github.com/EpiSci/oai-lte-5g-multi-ue-proxy

VII. Conclusion

We presented MAC-Gyver, an open MAC-layer framework for implementing scheduling dApps in the gNB. Its staged pipeline exposes self-contained interfaces to dApps, while its software-emulation component, `mac-emu`, executes the same policies with tens of UEs at real-time slot pace while emulating realistic channel dynamics. We demonstrate these capabilities with two dApp-based use cases that exercise different parts of the scheduler interface. First, proactive scheduling combines observer and candidate-request dApps through persistent context to implement closed-loop control across scheduling opportunities. In OTA experiments with sparse traffic, it reduces median RTT by approximately 50%. Second, we develop two real-time frequency-selective schedulers, which we first compare offline through simulations. Although offline analysis favors the more flexible design, executing both in MAC-Gyver (inside `mac-emu`) shows that the simpler policy performs better within the complete scheduling loop. This use case shows how MAC-Gyver supports algorithm design: offline and simulation-based analysis identifies potentially useful mechanisms, while execution in the complete scheduler determines which mechanisms affect end-to-end performance and should be retained. We then evaluate the retained policy further and show that it improves UL throughput under favorable channel conditions. Together, these applications show how MAC-Gyver supports policy design, controlled cell-scale evaluation, and OTA execution without reimplementation. The open-source release of MAC-Gyver and `mac-emu` supports reproducible AI-for-RAN research at the MAC layer.

References

[1] L. Kundu, X. Lin, R. Gadiyar, J.-F. Lacasse *et al.*, "Ai-ran: Transforming ran with ai-driven computing infrastructure," *IEEE Communications Magazine*, 2025.

[2] 3GPP, "NR; Medium Access Control (MAC) protocol specification," 3rd Generation Partnership Project (3GPP), Technical Specification (TS) 38.321, 2024, release 18, V18.x.

[3] A. Anand, G. de Veciana, and S. Shakkottai, "Joint Scheduling of URLLC and eMBB Traffic in 5G Wireless Networks," *Trans. on Net.*, vol. 28, no. 2, pp. 477–490, 2020.

[4] S. B. Lee, I. Pefkianakis, A. Meyerson, S. Xu *et al.*, "Proportional fair frequency-domain packet scheduling for 3GPP LTE uplink," in *Proc. IEEE INFOCOM*, 2009, pp. 2611–2615.

[5] F. Al-Tam, N. Correia, and J. Rodriguez, "Learn to Schedule (LEASCH): A Deep Reinforcement Learning Approach for Radio Resource Scheduling in the 5G MAC Layer," *IEEE Access*, 2020.

[6] P. Kela, B. Liu, and A. Valcarce, "From simulation to practice: Generalizable deep reinforcement learning for cellular schedulers," *arXiv preprint arXiv:2411.08529*, 2024.

[7] M. Polese, L. Bonati, S. D'Oro, S. Basagni *et al.*, "ColO-RAN: Developing machine learning-based xApps for open RAN closed-loop control on programmable experimental platforms," *IEEE Transactions on Mobile Computing*, vol. 22, no. 10, pp. 5787–5800, 2023.

[8] S. D'Oro, M. Polese, L. Bonati, H. Cheng *et al.*, "dApps: Distributed applications for real-time inference and control in O-RAN," *IEEE Communications Magazine*, vol. 60, no. 11, pp. 52–58, 2022.

[9] R. Barker, A. E. Dorcheh, T. Seyfi, and F. Afghah, "Real: Reinforcement learning-enabled xapps for experimental closed-loop optimization in o-ran with osc ric and srsran," in *ICC Workshops*, 2025.

[10] A. Lacava, L. Bonati, N. Mohamadi, R. Gangula *et al.*, "dApps: Enabling real-time AI-based open RAN control," *Com. Net.*, 2025.

[11] G. F. Riley and T. R. Henderson, "The ns-3 network simulator," in *Modeling and tools for network simulation*. Springer, 2010, pp. 15–34.

[12] M. Elkael, "Refactor DLSCH/ULSCH scheduler: Extract the proportional-fair policy behind a function-pointer interface," OpenAirInterface5G merge request !3902, 2026, merged 2026-05-23. [Online]. Available: https://gitlab.eurecom.fr/oai/openairinterface5g/-/merge_requests/3902

[13] M. Elkael, M. Polese, Y. Lee, K. Furueda *et al.*, "Auguste: Online-learning dapp for predictive urllc scheduling," *arXiv preprint*, 2026.

[14] M. Elkael, S. D'Oro, L. Bonati, M. Polese *et al.*, "Agentran: An agentic ai architecture for autonomous control of open 6g networks," *IEEE Communications Magazine*, 2026.

[15] P. F. Pérez, C. Fiandrino, and J. Widmer, "Characterizing and modeling mobile networks user traffic at millisecond level," in *Proceedings of the 17th ACM Workshop on Wireless Network Testbeds, Experimental evaluation & Characterization*, 2023, pp. 64–71.

[16] T. R. Henderson, M. Lacage, G. F. Riley, C. Dowell *et al.*, "Network simulations with the ns-3 simulator," *SIGCOMM demonstration*, vol. 14, no. 14, p. 527, 2008.

[17] D. Villa, I. Khan, F. Kaltenberger, N. Hedberg *et al.*, "X5G: An Open, Programmable, Multi-vendor, End-to-end, Private 5G O-RAN Testbed with NVIDIA ARC and OpenAirInterface," *IEEE Transactions on Mobile Computing*, 2025.

[18] L. Bonati, P. Johari, M. Polese, S. D'Oro *et al.*, "Colosseum: Large-scale wireless experimentation through hardware-in-the-loop network emulation," in *Proc. IEEE Int. Symp. on Dynamic Spectrum Access Networks (DySPAN)*, 2021.

[19] P. S. Upadhyaya, N. Tripathi, J. Gaeddert, and J. H. Reed, "Open ai cellular (oaic): An open source 5g o-ran testbed for design and testing of ai-based ran management algorithms," *IEEE Network*, 2023.

[20] C. Wei, A. Kak, N. Choi, and T. Wood, "5gperf: profiling open source 5g ran components under different architectural deployments," in *Proceedings of the ACM SIGCOMM Workshop on 5G and Beyond Network Measurements, Modeling, and Use Cases*, 2022, pp. 43–49.

[21] M. Rouili, N. Saha, M. Golkarifard, M. Zangooei *et al.*, "Evaluating open-source 5g sa testbeds: Unveiling performance disparities in ran scenarios," in *NOMS 2024-2024 IEEE Network Operations and Management Symposium*. IEEE, 2024, pp. 1–6.

[22] A. Mamaghani, U. Ghosh, S. Shakkottai, D. Bharadia *et al.*, "Tiny-twin: A cpu-native full-stack digital twin for nextg cellular networks," in *2026 IEEE International Symposium on Spectrum Innovation (DySPAN)*. IEEE, 2026, pp. 1–8.

[23] 3GPP, "Study on channel model for frequencies from 0.5 to 100 GHz," 3rd Generation Partnership Project (3GPP), Technical Report (TR) 38.901, 2026, release 19, V19.4.0.

[24] S. Lagen, K. Wanuga, H. Elkotby, S. Goyal *et al.*, "New Radio Physical Layer Abstraction for System-Level Simulations of 5G Networks," in *ICC 2020 - 2020 IEEE International Conference on Communications (ICC)*, 2020, pp. 1–7.

[25] Y. Deshpande, X. Wang, and W. Kellerer, "Openairlink: Reproducible wireless channel emulation using software defined radios," in *2024 IFIP Networking Conference (IFIP Networking)*. IEEE, 2024, pp. 1–6.

[26] 3GPP, "NR; Physical layer procedures for data," 3rd Generation Partnership Project (3GPP), Technical Specification (TS) 38.214, 2026, release 19, V19.4.0.

[27] U.S. Naval Research Lab., "Multi-generator (MGEN) network test tool," https://github.com/USNavalResearchLaboratory/mgen, accessed 2026.

[28] 3GPP, "Service requirements for the 5G system," 3rd Generation Partnership Project (3GPP), Technical Specification (TS) 22.261, 2026, release 19, V19.14.0.

[29] P. Popovski, J. J. Nielsen, C. Stefanovic, E. de Carvalho *et al.*, "Wireless Access for Ultra-Reliable Low-Latency Communications: Principles and Building Blocks," *IEEE Network*, vol. 32, no. 2, pp. 16–23, 2018.

[30] T.-K. Le, U. Salim, and F. Kaltenberger, "Enhancing URLLC Uplink Configured-Grant Transmissions," in *IEEE 93rd Vehicular Technology Conference (VTC2021-Spring)*, 2021.

[31] S. Maxenti, R. Shirkhani, M. Elkael, L. Bonati *et al.*, "AutoRAN: Automated and Zero-Touch Open RAN Systems," *arXiv preprint arXiv:2504.11233*, 2025.

[32] P. Ökvist, A. Simonsson, and H. Asplund, "LTE frequency selective scheduling performance and improvements assessed by measurements," in *Proc. IEEE 22nd Int. Symp. Personal, Indoor and Mobile Radio Communications (PIMRC)*, 2011, pp. 1919–1923.

[33] 3GPP, "NR; Physical layer procedures for control," 3rd Generation Partnership Project (3GPP), Technical Specification (TS) 38.213, 2025, release 18, V18.5.0.

[34] C. Y. Wong, R. S. Cheng, K. B. Letaief, and R. D. Murch, "Multiuser OFDM with adaptive subcarrier, bit, and power allocation," *IEEE Journal on Selected Areas in Communications*, vol. 17, no. 10, pp. 1747–1758, 1999.

[35] W. Rhee and J. M. Cioffi, "Increase in capacity of multiuser OFDM system using dynamic subchannel allocation," in *Proc. IEEE 51st Vehicular Technology Conference (VTC 2000-Spring)*, vol. 2, 2000, pp. 1085–1089.

[36] G. Ferreira, B. Bojović, A. A. Gargari, and S. Lágen, "SARA: Sub-band aware resource assignment for 5G-LENA," in *ICNS3*, 2025.

[37] A. Krishnamoorthy, P. J. Sartori, K. L. Baum, V. Nangia *et al.*, "Frequency semi-selective scheduling for a DFT-S-OFDM based uplink," in *Proc. IEEE 68th Vehicular Technology Conference (VTC 2008-Fall)*, 2008, pp. 1–5.

[38] F. D. Calabrese, P. E. Mogensen, C. Rosa, P.-H. Michaelsen *et al.*, "Search-tree based uplink channel aware packet scheduling for UTRAN LTE," in *Proc. IEEE 67th Vehicular Technology Conference (VTC 2008-Spring)*, 2008, pp. 1949–1953.

[39] A. Gutierrez, V. Rangel, J. Gomez, R. M. Edwards *et al.*, "A joint modulation-coding scheme and resource allocation in LTE uplink," *Elektronika ir Elektrotechnika*, vol. 26, no. 5, pp. 50–58, 2020.

[40] A. Domazetovic, L. J. Greenstein, N. B. Mandayam, and I. Seskar, "Estimating the doppler spectrum of a short-range fixed wireless channel," *IEEE Communications Letters*, vol. 7, no. 5, pp. 227–229, 2003.

[41] I. Rhee, M. Shin, S. Hong, K. Lee *et al.*, "On the lévy-walk nature of human mobility," *IEEE/ACM Transactions on Networking*, vol. 19, no. 3, pp. 630–643, 2011.

[42] F. Kaltenberger, A. P. Silva, A. Gosain, L. Wang *et al.*, "OpenAirInterface: Democratizing innovation in the 5G cellular ecosystem," *Computer Networks*, vol. 176, p. 107284, 2020.

[43] I. Gomez-Miguelez, A. Garcia-Saavedra, P. D. Sutton, P. Serrano *et al.*, "srsLTE: An open-source platform for LTE evolution and experimentation," in *WiNTECH*, 2016, pp. 25–32.

[44] M. Polese, L. Bonati, S. D'Oro, S. Basagni *et al.*, "Understanding O-RAN: Architecture, interfaces, algorithms, security, and research challenges," *IEEE Communications Surveys & Tutorials*, vol. 25, no. 2, pp. 1376–1411, 2023.

[45] R. Schmidt, M. Irazabal, and N. Nikaein, "FlexRIC: an SDK for next-generation SD-RANs," in *Proc. 17th Int. Conf. on Emerging Networking EXperiments and Technologies (CoNEXT)*, 2021, pp. 411–425.

[46] J. Breen, A. Buffmire, J. Duerig, K. Dutt *et al.*, "POWDER: Platform for open wireless data-driven experimental research," in *Proc. 14th Int. Workshop on Wireless Network Testbeds, Experimental Evaluation & Characterization (WiNTECH)*, 2020.

[47] T. R. Henderson, M. Lacage, G. F. Riley, C. Dowell *et al.*, "Network simulations with the ns-3 simulator," in *ACM SIGCOMM Conference Demonstration Session*, 2008.

[48] N. Patriciello, S. Lágen, B. Bojovic, and L. Giupponi, "An E2E simulator for 5G NR networks," *Simulation Modelling Practice and Theory*, vol. 96, p. 101933, 2019.

[49] J. Hoydis, S. Cammerer, F. Ait Aoudia, A. Vem *et al.*, "Sionna: An open-source library for next-generation physical layer research," *arXiv preprint arXiv:2203.11854*, 2022.

[50] NVIDIA, "NVIDIA Aerial CUDA-accelerated RAN," https://developer.nvidia.com/aerial, 2024, software and product documentation.

[51] B. Ryu, R. Knopp, M. Elkadi, D. Kim *et al.*, "5g-emane: Scalable open-source real-time 5g new radio network emulator with emane," in *MILCOM 2022-2022 IEEE Military Communications Conference (MILCOM)*. IEEE, 2022, pp. 553–558.

[52] M. Ghoshal, O. Basit, S. Wang, P. Dinh *et al.*, "A First Large-Scale Study of Operational 5G Standalone Networks," *CoNEXT*, 2025.

[53] R. M. Sohaib, S. T. Shah, O. Onireti, Y. Sambo *et al.*, "DRL-Based Joint Resource Scheduling of eMBB and URLLC in O-RAN," in *ICC Workshops*, 2024.

ACKNOWLEDGMENTS

We thank Robert Schmidt from OpenAirInterface for his precious comments during the review of our open-source contributions.